TGR-LORA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS

A Design Project Report
Presented to the School of Electrical and Computer Engineering of Cornell University
in Partial Fulfillment of the Requirements for the Degree of
Master of Engineering, Electrical and Computer Engineering

Submitted by
Beichen Yu (by325)
MEng Field Advisor: Adit Jain (aj457@cornell.edu), Vikram Krishnamurthy (vikramk@cornell.edu)
Degree Date: December 2026

Abstract

Master of Engineering Program
School of Electrical and Computer Engineering
Cornell University
Design Project Report

Project Title: TGR-LoRA: A CONTINUAL LEARNING PIPELINE FOR SMALL LANGUAGE MODELS
Author: Beichen Yu (by325)

Abstract:

This project implements and evaluates a parameter-efficient (PEFT) continual learning (CL) method designed for small language models (SLMs). It aims to address the problem of catastrophic forgetting, which means during training process across a sequence of different tasks, the parameter changes for learning later tasks might degrade the model's performance on previous tasks. Our project utilizes the Qwen2.5-3B-Instruct model and conducts experiments using five subsets from the TRACE continual learning enchmark, including C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds.

We implemented two distinct methods. Sequential LoRA serves as our baseline. It freezes the base model parameters and trains a single LoRA adapter across all tasks.

TGR/OrthoHist-LoRA is our method, which trains an active LoRA branch for the current task while integrating and consolidating the updated parameters into a frozen pool of historical bases and coefficients. In order to effectively use historical information, it also learns a low-rank mixing operator (defined as $M_t = I + U_t V_t^\top$) for the current task, thereby enabling the reuse of historical activation information. Furthermore, we implemented the metrics for evaluating continual learning performance, visualizations such as heatmaps showing knowledge reuse degree.

Executive Summary

The main objective of our project was to mitigate catastrophic forgetting while enabling subsequent tasks to leverage useful information acquired from prior tasks. Our system is entirely software-based. The core components include the Qwen2.5-3B-Instruct model, LoRA-based PEFT methods, TGR-LoRA method, and the TRACE benchmark.

Our proposed method involves **training an Active LoRA branch** tailored to the current task. Updated parameters from completed tasks **are stored in a frozen historical pool,** and each new task is equipped with a trainable low-rank mixing operator for reusing these historical activations. During the training process, both the base model and the historical pool remain frozen. There are 2 benefits: preventing the new training process from directly overwriting previously learned directions, while allowing subsequent tasks to reuse knowledge acquired during earlier tasks at the same time. In summary, the design philosophy is that the Active LoRA branch facilitates the model's exploration of new knowledge of the current task, while the low-rank mixing operator enables the efficient reuse of historical knowledge.

We successfully implemented a comprehensive, task-by-task training loop with checkpoint-based resume, task-level JSON logging, score matrix generation, Backward Transfer (BWT) calculation, average accuracy (AA) computation, and visualizations such as feature reuse heatmaps for the historical mixing operators.

During the project's initial poster presentation phase, reported results indicated more pronounced positive BWT scores than those subsequently reproduced in the final implementation. During the final code integration and validation phase, we refined our evaluation method, using to a method that computes task-by-task score matrices and BWT metrics.

Individual Contributions

Beichen Yu (by325): This is an individual project. I was responsible for the project's overall design, code implementation, debugging, experimental execution, poster creation, code repository organization, and final report writing. My specific contributions include:

- Defined the Continual Learning problem setting, selecting the Qwen2.5-3B-Instruct model and the TRACE benchmark as the foundation for experimental validation.
- Implemented a Sequential LoRA baseline model using the PEFT library, freezing the parameters of the Base Model during the implementation process.
- Implemented the TGR-LoRA layer wrapper module. The components include the frozen base model, an active LoRA branch, a frozen pool of historical basis and coefficients, an orthogonal gradient projection mechanism, and a low-rank mixer $M_t = I + U_t V_t^\top$, etc.
- Constructed a Task-by-Task Trainer, checkpoint saving logic, training resume logic, JSON logging function, a score matrix saving function, and a module for analyzing historical knowledge reuse.
- Implemented task-specific answer parsing functions and text generation evaluation metrics for five datasets: C-STANCE, FOMC, ScienceQA, NumGLUE-cm, and NumGLUE-ds to ensure an accurate evaluation method.
- Executed and debugged various experiments within Colab and AutoDL GPU environments, resolving technical challenges such as PyTorch/Transformers compatibility issues, CUDA architecture adaptation problems, and RTX 4090 GPU memory limitations.
- Organized and published the final GitHub code repository including the README and configuration files, and made detailed documentation to ensure the reproducibility of the experiments.

Contents

Chapter 1

Problem Statement and Requirements

1.1 Design Problem

Continual Learning aims to enable models to train on a continuous stream of incoming tasks. While learning a current task, a model must maintain its performance on previously learned tasks without degradation. Large Language Models (LLMs) are very susceptible to this challenge, as gradient updates applied to subsequent tasks may overwrite specific directions of model parameters that are critical for earlier tasks. The direct consequence of that is Catastrophic Forgetting.

This project focuses on PEFT supervised fine-tuning methods and Small Language Models (SLMs), as these methods are more lightweight and easier to deploy in real-world scenarios. We selected LoRA, as it achieves excellent results by freezing the base model and training only a set of low-rank adapter matrices. However, if a single LoRA adapter is trained sequentially across tasks, the model may still struggle to avoid the forgetting problem. While training a separate adapter for each task effectively prevents interference between tasks, this approach suffers from extremely low parameter reuse. Furthermore, during the inference phase, the corresponding adapter must be manually selected for each specific task, which can be complex to manage. Consequently, the core design challenge of this project is constructing a dynamically growing continual learning system that can both effectively preserve information of historical tasks and enable the efficient reuse of that information.

1.2 Requirements

The final requirements evolved iteratively over the two semesters. The focus of the first semester was to investigate the causes of catastrophic forgetting. Through a comprehensive literature review, we knew that catastrophic forgetting is most severe when the input spaces are similar but the output spaces differ. The work in the second semester focused on designing a robust training and evaluation pipeline with mathematical rigor. The finalized requirements are as follows:

1. **Sequential Task Training:** The system must train tasks sequentially in a fixed order, performing an evaluation after the completion of each task.
2. **Parameter Efficiency:** The Base Model must remain frozen, and only parameters associated with LoRA or TGR-LoRA are permitted to be trained.
3. **Baseline Comparison:** The system must employ Sequential LoRA as a fair comparative baseline.
4. **Historical Memory:** Update information (learned knowledge) generated by completed tasks must be stored within a fixed structure (served as memory).
5. **Historical Reuse:** Subsequent tasks should be able to reuse historical low-rank activations generated by previous tasks via a trainable mixing operator.
6. **Forgetting Metrics:** The system must compute and output a score matrix, average accuracy (AA), and the Backward Transfer (BWT) metric.
7. **Checkpoint Recovery:** If there is an interruption to the Colab or cloud GPU runtime, the training process must be capable of resuming from a task-level checkpoint.
8. **Reproducibility:** The project repository and report must include the complete codebase, execution commands, configuration files, explanations of output results, and environment setup instructions.
9. **Forgetting Mitigation:** The proposed method should effectively reduce the degree of forgetting and achieve a measurable level of Backward Transfer (BWT).

1.3 Practical Constraints

The project faced several constraints, these factors ultimately shaped our design approach.

- **GPU Memory.** The Qwen2.5-3B-Instruct model is substantial in size. Consequently, when performing TGR-LoRA training on an RTX 4090 card with a sequence length of 1024 (model generation) and a historical pool that continuously expands throughout the training process, GPU memory usage can result in an Out of Memory (OOM) error.
- **Cloud Runtime Environment.** Constrained by the resource limits of the Colab and the dynamic variability of the AutoDL environment, as well as the long duration of the training process, the project necessitated the task-level checkpointing to ensure the robustness of the training workflow.
- **Software Compatibility.** Compatibility among PyTorch, Transformers, PEFT, CUDA, and the underlying GPU architecture directly impacted experiments implementation. For instance, the RTX 5090 card requires the use of newer versions of the CUDA and PyTorch software stacks, so we finally choose 4090 for simpler configuration.

- **Evaluation Diversity.** The five TRACE tasks involve distinct types of responses. Therefore, it was necessary to design task-tailored functions for result parsing and the calculation of evaluation metrics.
- **Scope Reduction.** Although the project poster referenced the complete TRACE benchmark dataset, constraints regarding computational resources and time costs ultimately reduced our scope, with the final implementation of five representative tasks.

1.4 Final Design

Design	**Final implementation**
Backbone model	Qwen2.5-3B-Instruct
Task sequence	C-STANCE, FOMC, ScienceQA, NumGLUE-cm, NumGLUE-ds
Baseline	Sequential LoRA
Proposed method	TGR/OrthoHist-LoRA
Historical memory	Frozen QR-based basis-coefficient pool
Historical reuse	$M_t = I + U_t V_t^\top$ low-rank mixer
Metrics	Task score, average accuracy, BWT, score matrix
Artifacts	`score_matrix.png`, model checkpoints

Table 1.1: Final design specifications.

Chapter 2

Review of Alternatives and Selected Approach

2.1 Full Fine-Tuning

Full fine-tuning updates all of the model parameters. While this achieves the maximum flexibility, it is computationally expensive and therefore unsuitable for this project. Not only does it necessitates the storage of massive checkpoint files, but is also highly prone to severe catastrophic forgetting. Furthermore, this runs counter to our objective of achieving parameter efficiency.

2.2 Sequential LoRA

LoRA adds a low-rank adaptation branch to the original linear layer. It represents a trainable update to a linear layer as:

$$\Delta W = BA,$$

where $A \in \mathbb{R}^{r \times d_{\text{in}}}$ and $B \in \mathbb{R}^{d_{\text{out}} \times r}$. The forward residual is:

$$\Delta h = BAx.$$

The model's forward pass for a given input x is represented as the sum of the frozen base layer output and the low-rank adaptation:

$$h = W_0 x + \Delta W x = W_0 x + \frac{\alpha}{r} BAx, \tag{2.1}$$

where $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is the frozen pre-trained weight matrix, and $\frac{\alpha}{r}$ is a scaling factor defined by the LoRA rank r and the hyperparameter α. During training, only A and B receive gradient updates, while W_0 is frozen.

Sequential LoRA trains one adapter across tasks. It is simple and parameter-efficient, so it is the main baseline. Its weakness is that later task gradients may overwrite the same

LoRA matrices/important directions used by earlier tasks.

2.3 Adapter-per-Task

A separate LoRA adapter could be saved for each specific task. While this method can effectively minimize interference between tasks, it also avoids the sharing of knowledge across them. Furthermore, during the inference phase, this approach needs manual selection of task-specific adapter, which increases the complexity of management.

2.4 Fisher-SVD Design

During the initial phase of design exploration, we designed a method combining Fisher-weighted rank selection with Singular Value Decomposition (SVD)-based basis extraction. Specifically, once a new task is trained, we first employ Fisher information to assess the significance of the active LoRA rank

$$R_j = b_j \otimes a_j^\top = b_j a_j^\top$$

associated with that task. Then, we would utilize Fisher-weighted SVD to compress these critical directions and store them within a historical memory module. Finally, during the training of subsequent tasks, these previously saved directions would be protected. However, this implementation does not achieve a lossless compression of the knowledge acquired during each task, and Fisher information can be accurately estimated only when the model has reached the optimal training state of current task.

2.5 Final Selected Approach: TGR/OrthoHist-LoRA

Our final implementation is designed to mitigate catastrophic forgetting while allowing controlled reuse of historical knowledge. It consists of the following components:

1. **Frozen Base Model:** The original Qwen2.5-3B-Instruct model is frozen. All low-rank adaptation occurs in separate trainable branches.

2. **Active LoRA Branch:** For each new task, a dedicated low-rank adapter is trained. The forward computation for a wrapped linear layer is:

$$h' = W_0 x + \frac{\alpha}{r}(\Delta h_{\text{active}} + \Delta h_{\text{history}})$$

   where the active task contribution is

$$\Delta h_{\text{active}} = B_{\text{active}} A_{\text{active}} x.$$

The active branch captures task-specific updates without modifying the frozen base weights.

3. **Historical Basis-Coefficient Pool:** At the end of each task, the active update is committed into a frozen historical pool using QR factorization:

$$B_{\text{active}}A_{\text{active}} = Q(RA_{\text{active}}),$$

where Q (basis) and RA_{active} (coefficients) are stored as historical memory. The coefficients represent how input information is projected into the low-rank feature space (rank-dimension), while the basis maps these features back to the output dimension to modify the model response (output). Both Q and RA_{active} are frozen and can be reused in later tasks. We use QR decomposition to ensure that the model learns differenct directions/aspects of knowledge for a task, also laying the foundation for gradient orthogonal projection during the training of subsequent tasks.

4. **Low-Rank Historical Reuse:** For a new task t, all previous basis and coefficient blocks are concatenated:

$$B_{\text{old}} = [Q_1, Q_2, \ldots, Q_{t-1}], \quad C_{\text{old}} = \begin{bmatrix} C_1 \\ C_2 \\ \vdots \\ C_{t-1} \end{bmatrix}.$$

A trainable low-rank mixing matrix $M_t = I + U_t V_t^\top$ is applied to flexibly combine historical contributions:

$$\Delta h_{\text{history}} = B_{\text{old}} M_t C_{\text{old}} x.$$

This allows the model to reuse previous task knowledge while keeping it separate from the current task updates. mt_k denotes the rank of the mixing matrix M_t, which determines the low-rank dimension for linearly combining historical bases from the Basis Pool. To ensure parameter efficiency, M_t is decomposed into two matrices: $M_{t,U} \in \mathbb{R}^{(r \cdot i) \times mt_k}$ and $M_{t,V} \in \mathbb{R}^{(r \cdot i) \times mt_k}$ (i represents the number of historical tasks). By constraining M_t to a lower rank, the model can combine those features that are most significant from past tasks and possess strong generalization capabilities.

5. **Orthogonal Gradient Hook/Projection:** To prevent interference between new task learning and previously stored historical directions, a backward hook is applied on the active LoRA branch. This hook projects gradients of the active branch onto the subspace orthogonal to the historical basis. So the model's output direction of the new task is orthogonal to all historical tasks' output directions. To be specific, if G_{active} is the gradient for the active LoRA B matrix, the hook computes:

$$G_{\text{active}} \leftarrow G_{\text{active}} - Q_{\text{old}}(Q_{\text{old}}^\top G_{\text{active}})$$

where Q_{old} is the concatenated basis from all previous tasks. This ensures that updates

for the current task do not overwrite directions important for historical tasks, effectively reducing catastrophic forgetting.

In summary, our design separates old-task memory from new-task residual learning, allows controlled low-rank reuse, and incorporates orthogonal gradient projection to preserve historical knowledge while adapting to new tasks. During the training process, the base model and historical pool are frozen, while the low-rank mixing matrix and acitve LoRA are trainable as shown in the following figure.

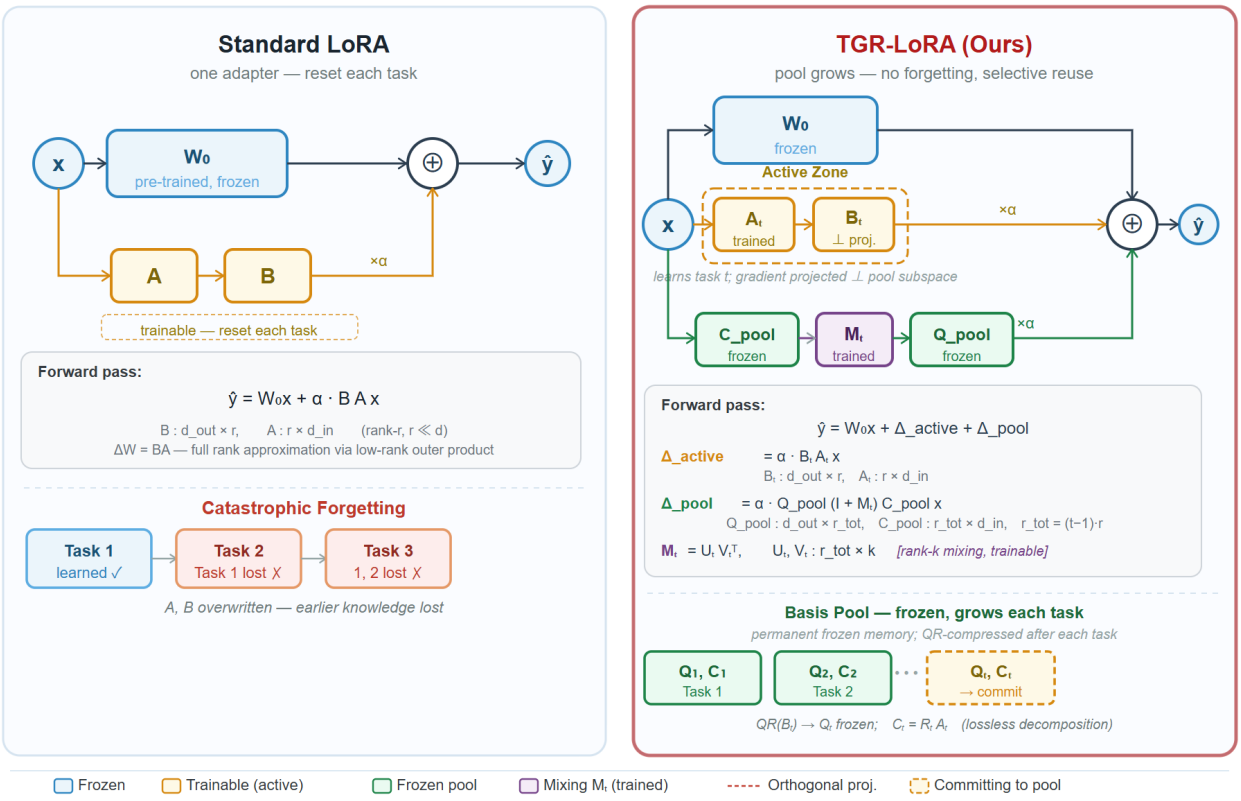

Figure 2.1: Final Deisgn

TGR-LoRA Algorithm

Algorithm 1: TGR-LoRA Continual Learning

Input: Tasks $\{T_1, \ldots, T_N\}$; LoRA rank r; scaling α; mt-rank k
Output: Model + frozen Basis Pool

for *each task* $t = 1$ *to* N **do**
  // Step 1 -- Initialize Active Zone
  $A_t \sim$ Kaiming-uniform
  if $t = 1$ **then**
    $B_t \leftarrow \mathbf{0}$ // standard LoRA cold start
  else
    Sample $\tilde{B} \sim \mathcal{N}(0, I)$
    $B_t \leftarrow \tilde{B} - Q_{\text{pool}}\left(Q_{\text{pool}}^{\top}\tilde{B}\right)$
    $B_t \leftarrow \text{QR}(B_t)$ // init orthogonal to all pool bases
  end
  // Step 2 -- Initialize Mixing Matrix M_t
  $r_{\text{tot}} \leftarrow (t-1) \cdot r$
  $U_t, V_t \in \mathbb{R}^{r_{\text{tot}} \times k}$, zero-pad from M_{t-1} (warm start)
  // Step 3 -- Training Loop
  while *not converged* **do**
    Forward: $\hat{y} = W_0 x + \Delta W_t(x)$
    Backward, then project gradient:
      $g \leftarrow \nabla_{B_t} \mathcal{L}$
      for $j = 1$ *to* $t-1$ **do**
        $g \leftarrow g - Q_j\left(Q_j^{\top} g\right)$ // Gram-Schmidt onto $\text{span}(Q_j)^{\perp}$
      end
      Apply projected gradient g to B_t
    Update A_t, U_t, V_t (no projection)
  end
  // Step 4 -- Commit to Basis Pool
  $[Q_t, R_t] \leftarrow \text{QR}(B_t)$ // $Q_t^{\top} Q_t = I$; lossless factorisation
  $C_t \leftarrow R_t A_t$ // $Q_t C_t x = B_t A_t x$ preserved exactly
  Freeze $(Q_t, C_t) \rightarrow$ Basis Pool
  Freeze and archive M_t
  Reset: $B_t \leftarrow \mathbf{0}$,
  $A_t \sim$ Kaiming
end

Figure 2.2: TGR-LoRA Overall Algorithm

Chapter 3

System Design and Implementation

3.1 System Overview

Our project consists of four main components. First, the model and tokenizer are loaded from Hugging Face. Second, PEFT LoRA and TGR-LoRA is injected into selected linear modules (q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj for Qwen-2.5-3B-Instruct). Next, the trainer processes the tasks sequentially, saving a checkpoint and performing an evaluation upon the completion of each task. Finally, the analysis tool saves the score matrix and generates plots.

3.2 Repository Structure

Our repository structure and related content is as follows.

File	Purpose
`main.py`	Loads configuration, model, tokenizer, and related trainer.
`run_experiments.sh`	Runs Sequential LoRA and selected TGR-LoRA configurations.
`summarise_results.py`	Reads run histories and prints summary metrics.
`sequential_lora.yaml`	Sequential LoRA configuration.
`ortho_hist_k4.yaml`	TGR-LoRA $mtk = 4$ configuration.
`ortho_hist_k8.yaml`	TGR-LoRA $mtk = 8$ configuration.
`ortho_hist_k16.yaml`	TGRTGR-LoRA $mtk = 16$ configuration.
`config.py`	Experiment Configuration and validation.
`data_trace.py`	TRACE dataset loading and collation.
`eval_trace.py`	Task-specific answer parsing and scoring.
`trainers.py`	TGR-LoRA and Sequential LoRA trainers.
`ortho_hist_lora.py`	TGR-LoRA layer and manager implementation.
`checkpointing.py`	Task-level checkpoint saving and loading.
`analysis_utils.py`	Score matrix and reuse heatmap generation.

Table 3.1: Main repository components.

3.3 Sequential LoRA Baseline Implementation

Sequential LoRA is implemented using the PEFT framework. The base Qwen model remains frozen, with only the LoRA A and B matrices being trainable during the training process. The same LoRA adapter is utilized across all tasks. This serves as the benchmark to assess whether Sequential LoRA suffers from catastrophic forgetting of earlier tasks.

The number of trainable parameters in this benchmark is approximately 29.9 million. This aligns with the parameter count obtained when applying a rank-16 LoRA to the attention projection and MLP projection layers of the Qwen2.5-3B-Instruct model, which validates that the base model freezing logic has been successfully applied.

3.4 TGR-LoRA Implementation

The TGR-LoRA wrapper replaces selected `nn.Linear` modules. Each wrapper stores:

- the frozen base linear layer;
- the active LoRA matrices `lora_A` and `lora_B`;
- a frozen `basis_pool` containing previous basis and coefficient blocks;

- the current task mixer matrices `mt_U` and `mt_V`;
- a backward hook that projects active `lora_B` gradients away from historical basis directions.

At the beginning of task 1, the current active branch is initialized the same as standard LoRA. In subsequent tasks, the active LoRA B is initialized orthogonally to the historical basis pool. Once the training of current task is finished, the current active update will be QR decomposed and store into the historical pool.

Chapter 4

Testing and Experimental Validation

4.1 Evaluation Method

The final implementation evaluates the model after the completion/training of each task. Each row of the score matrix corresponds to the model after the completion of training for a task, while each column corresponds to a task being evaluated. The training sequence for the final five tasks is as follows:

$$\text{C-STANCE} \rightarrow \text{FOMC} \rightarrow \text{ScienceQA} \rightarrow \text{NumGLUE-cm} \rightarrow \text{NumGLUE-ds}.$$

The task evaluation metrics are:

- C-STANCE: accuracy
- FOMC: accuracy
- ScienceQA: accuracy
- NumGLUE-cm: exact match
- NumGLUE-ds: exact match

The continual learning metrics are:

$$\text{AA}_t = \frac{1}{t}\sum_{i=1}^{t} S_{t,i},$$

and

$$\text{BWT}_t = \frac{1}{t-1}\sum_{i=1}^{t-1}(S_{t,i} - S_{i,i}),$$

where $S_{t,i}$ is the score on task i after training through task t.

We choose two classical continual learning metrics. AA_t represents the model's average performance across all trained tasks after completing the training for the first t tasks. This metric reflects the model's overall performance as it continuously acquires new knowledge.

BWT_t illustrates the impact of learning new tasks on previously learned tasks. By calculating the difference between a past task's score at the current moment and its score after the first training, it measures whether the model has suffered from catastrophic forgetting.

4.2 Sequential LoRA Five-Task Baseline

Table 4.1 reports the performance of the Sequential LoRA baseline. The revised results demonstrate that Sequential LoRA serves as a robust baseline in this experiment. Upon completing training for the FOMC task, its C-STANCE score rose slightly from 0.6000 to 0.6100, yielding a corresponding BWT value of +0.0100. Following the subsequent completion of the ScienceQA and NumGLUE-cm tasks, the baseline's performance remained stable. After finishing the NumGLUE-cm task, its AA score reached 0.7051, with a BWT value of +0.0300.

Upon completing the final task NumGLUE-ds, Sequential LoRA achieved a final AA score of 0.6393 and a final BWT value of −0.0234, which shows a slight forgetting degree. However, we also note that this outcome is decided according to the experimental configuration: when we set the model's maximum generation length to just 512 with 500 training samples, the degree of forgetting observed in Sequential LoRA was relatively low. However, when this limit was increased from 512 to 1024, and the number of training samples increased from 500 to 1000, the model exhibited more serious forgetting.

Table 4.1: Sequential LoRA Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA ↑	BWT ↑	FWT ↑
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.6100	0.7300	–	–	–	0.6700	+0.0100	0.0000
ScienceQA	0.6000	0.7300	0.8200	–	–	0.7167	0.0000	0.0000
NumGLUE-cm	0.6200	0.7500	0.8700	0.5802	–	0.7051	+0.0300	0.0000
NumGLUE-ds	0.6100	0.7100	0.6500	0.6667	0.5600	0.6393	-0.0234	0.0000

4.3 TGR-LoRA Five-Task Comparison

For TGR-LoRA, we evaluated three settings for the historical mixing rank: $mt\_k = 4$, $mt\_k = 8$, and $mt\_k = 16$. This mixing rank represents the size of the historical reuse operator for each task.

A larger mt_k gives the model greater degrees of freedom to recombine historical low-rank activations, while simultaneously introducing more trainable parameters into the reuse path.

The final results indicate that, in the five-task setting, $mt\_k = 8$ achieved the best final AA and BWT scores, yielding a final AA of 0.6296 and a final BWT of −0.0400. Furthermore, across various configurations, the model's degree of forgetting consistently followe a pattern of firstly increasing and then decreasing. This trend also persisted after we increased the maximum generation length and the number of training samples. Moreover, under this configuration, the performance gap between TGR-LoRA and Sequential LoRA was smaller.

4.3.1 TGR-LoRA with $mt\_k = 4$

Table 4.2 presents the performance of TGR-LoRA under the $mt\_k = 4$ configuration. This configuration exhibited the most severe early forgetting phenomenon. Upon completion of the FOMC task, the C-STANCE score dropped from 0.6000 to 0.4400, corresponding to a BWT value of −0.1600. Subsequent tasks led to a partial recovery of the model's performance: After completing the ScienceQA task, the BWT improved to −0.0300, while after completing the NumGLUE-cm task, the BWT briefly turned positive, reaching +0.0100. However, upon the completion of the final NumGLUE-ds task, the final AA value was 0.6182, while the final BWT declined to −0.0498. Therefore, the $mt\_k = 4$ configuration failed to provide a stable and effective mitigation of forgetting.

Table 4.2: TGR-LoRA ($mt\_k = 4$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA ↑	BWT ↑	FWT ↑
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.4400	0.7100	–	–	–	0.5750	-0.1600	0.0000
ScienceQA	0.5700	0.6800	0.8200	–	–	0.6900	-0.0300	0.0000
NumGLUE-cm	0.5900	0.7100	0.8600	0.5802	–	0.6851	+0.0100	0.0000
NumGLUE-ds	0.5700	0.7100	0.7000	0.5309	0.5800	0.6182	-0.0498	0.0000

4.3.2 TGR-LoRA with $mt\_k = 8$

Table 4.3 presents the performance of TGR-LoRA under the $mt\_k = 8$ configuration, which represents the strongest-performing TGR-LoRA configuration. Upon completion of the FOMC task, the C-STANCE score dropped from 0.6000 to 0.4900, yielding a corresponding BWT value of −0.1100. This result still indicates the early forgetting problem. However, its performance in subsequent stages was more robust than that of the $mt\_k = 4$ setting. After completing the ScienceQA task, the BWT improved to −0.0400. Upon the completion of the NumGLUE-cm task, the BWT further improved to −0.0200. And after the entire sequence of five tasks, the $mt\_k = 8$ configuration finally achieved an AA score of 0.6296 and a BWT value of −0.0400.

Table 4.3: TGR-LoRA ($mt\_k = 8$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA ↑	BWT ↑	FWT ↑
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.4900	0.7800	–	–	–	0.6350	-0.1100	0.0000
ScienceQA	0.5500	0.7500	0.7900	–	–	0.6967	-0.0400	0.0000
NumGLUE-cm	0.5800	0.7300	0.8000	0.5679	–	0.6695	-0.0200	0.0000
NumGLUE-ds	0.5600	0.7400	0.7100	0.5679	0.5700	0.6296	-0.0400	0.0000

4.3.3 TGR-LoRA with $mt\_k = 16$

Table 4.4 reports the performance of TGR-LoRA with $mt\_k = 16$. Among all tested configurations, this setting employs the largest historical mixing rank (the same as the LoRA rank 16). During the second-stage tasks, it exhibited less early forgetting compared to the $mt\_k = 4$ and $mt\_k = 8$ settings. Upon completion of the FOMC task, the C-STANCE metric dropped from 0.6000 to 0.5100, yielding a corresponding BWT value of -0.0900. However, this advantage did not persist throughout the entire task sequence. After completing the ScienceQA task, the BWT shifted to -0.0750. Upon finishing the NumGLUE-ds task, the final BWT changed to -0.0687, which was the weakest final BWT score among all TGR-LoRA configurations. This suggests that simply increasing the mixing rank does not always guarantee improved reuse of historical information.

Table 4.4: TGR-LoRA ($mt\_k = 16$) Performance

Trained Up To	C-ST	FOMC	SciQ	Num-cm	Num-ds	AA ↑	BWT ↑	FWT ↑
C-STANCE	0.6000	–	–	–	–	0.6000	0.0000	0.0000
FOMC	0.5100	0.7500	–	–	–	0.6300	-0.0900	0.0000
ScienceQA	0.5100	0.6900	0.8200	–	–	0.6733	-0.0750	0.0000
NumGLUE-cm	0.5500	0.7100	0.8000	0.5802	–	0.6601	-0.0367	0.0000
NumGLUE-ds	0.5600	0.7100	0.6500	0.5556	0.6000	0.6151	-0.0687	0.0000

4.3.4 Final Method Comparison

Table 4.5 shows a comparison of the final performance. Across all completed experiments, Sequential LoRA achieved the best final AA and BWT scores. Among the TGR-LoRA configurations, the configuration with $mt\_k = 8$ performed the best, followed by $mt\_k = 4$ and $mt\_k = 16$.

Consequently, the final results are mixed. Although the TGR-LoRA successfully incorporates historical memory capabilities and trainable reuse of historical information, it currently fails to consistently outperform Sequential LoRA. Furthermore, likely due to the significant difference between the five selected datasets, the current model has only minimal reuse of historical task information at each layer (with values less than 0.001). We also observed that as the number of training samples increases, the performance gap between TGR-LoRA and Sequential LoRA narrows. Therefore, our method appears to be better suited for scenarios of high task similarity and larger training samples, as well as larger model generation length. Future directions should explore the integration of task-aware retrieval or task-specific inference mechanisms to ensure that, when evaluating prior tasks, the system utilizes the historical states most relevant to them.

Table 4.5: Final Comparison After the Five-Task Sequence

Method	C-ST	FOMC	SciQ	Num-cm	Num-ds	Final AA ↑	Final BWT ↑
Sequential LoRA	0.6100	0.7100	0.6500	0.6667	0.5600	0.6393	-0.0234
TGR-LoRA ($mt\_k = 4$)	0.5700	0.7100	0.7000	0.5309	0.5800	0.6182	-0.0498
TGR-LoRA ($mt\_k = 8$)	0.5600	0.7400	0.7100	0.5679	0.5700	0.6296	-0.0400
TGR-LoRA ($mt\_k = 16$)	0.5600	0.7100	0.6500	0.5556	0.6000	0.6151	-0.0687

Chapter 5

Reproducibility, User Manual, and Bill of Materials

5.1 Code Repository

The final code repository is available at https://github.com/by325-hub/ECE-5996-design-proj.git or https://github.com/byu959397-cell/ECE-5996-design-proj-code.git if the previous one is not accessible.

The repository contains the Python source files, YAML configuration files, run scripts, README, and result artifact instructions.

5.2 Environment Setup

Listing 5.1: Basic environment setup.

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

For AutoDL or Colab, the following commands were useful:

Listing 5.2: Cloud environment notes.

```
export HF_ENDPOINT="https://hf-mirror.com"
pip uninstall -y torchao
export PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128
```

5.3 Dataset Layout

Listing 5.3: Expected TRACE-style dataset layout.

```
TRACE/
  C-STANCE/
    train.json
    eval.json
```

```
    test.json
  FOMC/
    train.json
    eval.json
    test.json
  ScienceQA/
    train.json
    eval.json
    test.json
  NumGLUE-cm/
    train.json
    eval.json
    test.json
  NumGLUE-ds/
    train.json
    eval.json
    test.json
```

5.4 Running Experiments

Listing 5.4: Run Sequential LoRA only.

```
python main.py --config configs/sequential_lora.yaml
```

Listing 5.5: Run OrthoHist-LoRA k=4 only.

```
python main.py --config configs/ortho_hist_k4.yaml
```

Listing 5.6: Run Sequential LoRA and OrthoHist-LoRA through the shell script.

```
bash run_experiments.sh 0 4
```

Listing 5.7: Run only OrthoHist-LoRA and skip Sequential LoRA.

```
RUN_SEQ=0 bash run_experiments.sh 0 4
```

5.5 Expected Outputs

Each run writes to the output directory defined in its YAML file. Important files are:

- `history.json`: task-level scores, AA, BWT, and FWT.
- `score_matrix_raw.json`: raw score matrix.
- `analysis/score_matrix.png`: score matrix heatmap.
- `analysis/reuse_heatmap.png`: OrthoHist reuse heatmap.
- `logs/events.jsonl`: JSONL training and evaluation log.
- `checkpoints/task_XX/DONE`: task completion marker for resume.

5.6 Software Bill of Materials

Item	Purpose	Cost
Python	Main programming language	$0
PyTorch	Tensor computation and model training	$0
Transformers	Qwen model and tokenizer loading	$0
PEFT	Sequential LoRA baseline	$0
NumPy, SciPy	Analysis and numerical utilities	$0
Matplotlib	Score matrix and reuse heatmap visualization	$0
TRACE-style dataset	Continual learning validation	$0
Cloud GPU compute	Colab / AutoDL training and debugging	Variable

Table 5.1: Software bill of materials.

Bibliography

[1] E. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," 2021.

[2] TRACE Benchmark. https://github.com/BeyonderXX/TRACE.git

[3] Qwen Team. https://huggingface.co/Qwen/Qwen2.5-3B-Instruct

[4] ChatGPT and Gemini were utilized to assist in polishing portions of the text, debugging and optimizing the code, and evaluating the TRACE evaluation logic. The core concepts and strategies of all the proposed solutions and ideas are the author's.

Appendix A

Appendix A: Code Listings

A.1 Project Entry Point

Listing A.1: main.py: Project Entry Point

```python
#!/usr/bin/env python3
import argparse
import pathlib

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

from src.config import load_config
from src.ortho_hist_lora import OrthoHistManager
from src.trainers import ContinualTrainer, SequentialLoRATrainer, set_seed


def build_model_and_tokenizer(cfg):
    print(f"Loading {cfg.model_name}")

    tok = AutoTokenizer.from_pretrained(
        cfg.model_name,
        use_fast=cfg.use_fast_tokenizer,
        padding_side=cfg.padding_side,
        trust_remote_code=True,
    )
    if tok.pad_token is None:
        tok.pad_token = tok.eos_token

    dtype = torch.bfloat16 if cfg.bf16 else torch.float32
    model = AutoModelForCausalLM.from_pretrained(
        cfg.model_name,
        torch_dtype=dtype,
        trust_remote_code=True,
    )

    # Training does not need KV cache. This also avoids warnings in some models.
    if hasattr(model, "config"):
        model.config.use_cache = False

    return model, tok


```

```python
def inject_peft_lora(model, cfg):
    from peft import LoraConfig, TaskType, get_peft_model

    lora_cfg = LoraConfig(
        r=cfg.rank,
        lora_alpha=cfg.alpha,
        lora_dropout=cfg.dropout,
        target_modules=cfg.target_modules,
        task_type=TaskType.CAUSAL_LM,
        bias="none",
    )
    return get_peft_model(model, lora_cfg)


def parse_args():
    parser = argparse.ArgumentParser()
    parser.add_argument("--config", required=True)
    parser.add_argument("--output_dir", default=None)
    parser.add_argument("--method", default=None)
    parser.add_argument("--mt_rank", type=int, default=None)
    parser.add_argument("--rank", type=int, default=None)
    return parser.parse_args()


def apply_cli_overrides(cfg, args):
    if args.output_dir is not None:
        cfg.output_dir = args.output_dir
    if args.method is not None:
        cfg.method = args.method
    if args.mt_rank is not None:
        cfg.mt_rank = args.mt_rank
    if args.rank is not None:
        cfg.rank = args.rank
    return cfg


def main():
    args = parse_args()
    cfg = apply_cli_overrides(load_config(args.config), args)

    pathlib.Path(cfg.output_dir).mkdir(parents=True, exist_ok=True)

    set_seed(cfg.seed)
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    print(f"device: {device}")
    print(f"method: {cfg.method}")
    print(f"output_dir: {cfg.output_dir}")
    print(f"tasks: {', '.join(cfg.train_tasks)}")

    model, tok = build_model_and_tokenizer(cfg)
    model.to(device)

    if cfg.method == "ortho_hist":
        manager = OrthoHistManager(
            model,
            target_modules=cfg.target_modules,
            rank=cfg.rank,
            alpha=cfg.alpha,
            dropout=cfg.dropout,
            mt_rank=cfg.mt_rank,
```

```
            device=device,
        )
        manager.inject()
        ContinualTrainer(cfg, tok, model, manager, device).train_all_tasks()

    elif cfg.method == "sequential_lora":
        model = inject_peft_lora(model, cfg)
        model.to(device)
        SequentialLoRATrainer(cfg, tok, model, device).train_all_tasks()

    else:
        raise ValueError(f"Unknown method: {cfg.method}")

    print(f"\nDone. Results in {cfg.output_dir}")


if __name__ == "__main__":
    main()
```

A.2 Experiment Runner

Listing A.2: run_experiments.sh: Experiment Runner

```
#!/bin/bash
# run_experiments.sh
# Run Sequential LoRA and OrthoHist-LoRA on the five TRACE tasks.
# Usage:
#   bash run_experiments.sh [gpu_id] [k ...]
# Examples:
#   bash run_experiments.sh 0          # run k=4,8,16 if configs exist
#   bash run_experiments.sh 0 8        # run only k=8
#   bash run_experiments.sh 0 4 8 16

set -euo pipefail

GPU=${1:-0}
shift || true

KS=("$@")
if [ ${#KS[@]} -eq 0 ]; then
    KS=(4 8 16)
fi

export CUDA_VISIBLE_DEVICES="$GPU"
CONFIG_DIR=${CONFIG_DIR:-configs}

RUN_SEQ=${RUN_SEQ:-1}
RUN_ORTHO=${RUN_ORTHO:-1}

run_config() {
    local label="$1"
    local cfg="$2"

    if [ ! -f "$cfg" ]; then
        echo "[Skip] $label: config not found: $cfg"
        return 0
    fi

```

```
    echo ""
    echo ">>> $label"
    echo "    config: $cfg"
    python main.py --config "$cfg"
}


echo "===== Continual LoRA Experiments ====="
echo "GPU: $GPU"
echo "CONFIG_DIR: $CONFIG_DIR"
echo "OrthoHist k values: ${KS[*]}"
echo ""

if [ "$RUN_SEQ" = "1" ]; then
    run_config "Sequential LoRA" "$CONFIG_DIR/sequential_lora.yaml"
else
    echo "[Skip] Sequential LoRA disabled by RUN_SEQ=0"
fi

if [ "$RUN_ORTHO" = "1" ]; then
    for K in "${KS[@]}"; do
        run_config "OrthoHist-LoRA mt_rank=$K" "$CONFIG_DIR/ortho_hist_k${K}.yaml"
    done
else
    echo "[Skip] OrthoHist-LoRA disabled by RUN_ORTHO=0"
fi


echo ""
echo "===== Finished ====="
echo "To generate the table:"
echo "  python summarise_results.py"
```

A.3 Result Summary Script

Listing A.3: summarise_results.py: Result Summary Script

```
#!/usr/bin/env python3
from __future__ import annotations

import json
from pathlib import Path
from typing import Any, Dict, Optional

TASKS = [
    "C-STANCE",
    "FOMC",
    "ScienceQA",
    "NumGLUE-cm",
    "NumGLUE-ds",
]

SHORT = ["C-ST", "FOMC", "SciQ", "Num-cm", "Num-ds"]

RUNS = {
    "Sequential LoRA": "outputs/sequential_lora",
    "OrthoHist k=4": "outputs/ortho_hist_k4",
    "OrthoHist k=8": "outputs/ortho_hist_k8",
```

```
    "OrthoHist k=16": "outputs/ortho_hist_k16",
}


def load_json(path):
    if not path.exists():
        return None
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)


def load_history(run_dir: str):
    return load_json(Path(run_dir) / "history.json")


def final_row_from_history(history):
    if not history:
        return {
            "scores": {},
            "aa": None,
            "bwt": None,
            "fwt": None,
        }

    last = history[-1]

    # New trainer format.
    scores = last.get("seen_scores")
    aa = last.get("average_accuracy")

    # Backward compatibility with older SequentialLoRATrainer history.
    if scores is None:
        scores = last.get("scores", {})
    if aa is None:
        aa = last.get("aa")

    return {
        "scores": scores or {},
        "aa": aa,
        "bwt": last.get("bwt"),
        "fwt": last.get("fwt"),
    }


def fmt(v):
    return "" if v is None else f"{v * 100:.1f}"


def build_rows():
    rows = []
    for name, run_dir in RUNS.items():
        history = load_history(run_dir)
        row = {"name": name, "run_dir": run_dir}
        row.update(final_row_from_history(history))
        rows.append(row)
    return rows


def print_console_table(rows):
    header = (
        f"{'Method':<22}"
```

```
        + "".join(f"{s:>8}" for s in SHORT)
        + f"{'AA':>8}{'BWT':>8}{'FWT':>8}"
    )
    print(header)
    print("-" * len(header))

    for row in rows:
        line = f"{row['name']:<22}"
        for task in TASKS:
            line += f"{fmt(row['scores'].get(task)):>8}"
        line += f"{fmt(row['aa']):>8}"
        line += f"{fmt(row['bwt']):>8}"
        line += f"{fmt(row['fwt']):>8}"
        print(line)

    missing = [r for r in rows if r["aa"] is None]
    if missing:
        print("\nMissing runs:")
        for r in missing:
            print(f"  - {r['name']}: {r['run_dir']}/history.json not found")

    print("\nValues are percentages. BWT < 0 indicates forgetting.")


def print_latex_table(rows):
    print(r"\begin{table}[t]")
    print(r"\centering")
    print(r"\small")
    cols = "l" + "c" * (len(TASKS) + 3)
    print(r"\begin{tabular}{" + cols + r"}")
    print(r"\toprule")
    print("Method & " + " & ".join(SHORT) + r" & AA & BWT & FWT \\")
    print(r"\midrule")

    for row in rows:
        cells = [row["name"]]
        for task in TASKS:
            cells.append(fmt(row["scores"].get(task)))
        cells.append(fmt(row["aa"]))
        cells.append(fmt(row["bwt"]))
        cells.append(fmt(row["fwt"]))
        print(" & ".join(cells) + r" \\")

    print(r"\bottomrule")
    print(r"\end{tabular}")
    print(
        r"\caption{Continual learning results on five TRACE tasks "
        r"(Qwen2.5-3B-Instruct, LoRA rank=16). "
        r"AA denotes average accuracy over seen tasks after the full task sequence. "
        r"BWT$<$0 indicates forgetting.}"
    )
    print(r"\label{tab:main_results}")
    print(r"\end{table}")


def main():
    rows = build_rows()
    print_console_table(rows)
    print()
    print_latex_table(rows)

```

```
144
145 if __name__ == "__main__":
146     main()
```

A.4 Python Requirements

Listing A.4: requirements.txt: Python Requirements

```
1  torch>=2.1.0
2  transformers>=4.40.0
3  peft>=0.10.0
4  accelerate>=0.27.0
5
6  safetensors>=0.4.2
7  huggingface_hub>=0.23.0
8  tokenizers>=0.19.0
9  packaging>=23.0
10
11 numpy>=1.24.0
12 pyyaml>=6.0
13 tqdm>=4.65.0
14 matplotlib>=3.7.0
15 scipy>=1.10.0
16 datasets>=2.18.0
17 rouge-score>=0.1.2
```

A.5 Configuration: Sequential LoRA

Listing A.5: configs/sequential_lora.yaml: Configuration: Sequential LoRA

```
1  seed: 42
2  output_dir: ./outputs/sequential_lora
3  trace_root: ./TRACE
4  model_name: Qwen/Qwen2.5-3B-Instruct
5  use_fast_tokenizer: false
6  padding_side: left
7  max_length: 1024
8  per_device_batch_size: 2
9  grad_accum_steps: 4
10 num_epochs: 3
11 learning_rate: 0.0002
12 weight_decay: 0.0
13 warmup_ratio: 0.03
14 max_grad_norm: 1.0
15 bf16: true
16 compile_model: false
17 log_every: 10
18 eval_every_epoch: false
19 save_every_task: true
20 method: sequential_lora
21 rank: 16
22 alpha: 32
23 dropout: 0.0
24 debug_eval: false
25 target_modules:
```

```
- q_proj
- k_proj
- v_proj
- o_proj
- gate_proj
- up_proj
- down_proj
train_tasks:
- C-STANCE
- FOMC
- ScienceQA
- NumGLUE-cm
- NumGLUE-ds
eval_tasks:
- C-STANCE
- FOMC
- ScienceQA
- NumGLUE-cm
- NumGLUE-ds
```

A.6 Configuration: OrthoHist-LoRA k=4

Listing A.6: configs/ortho_hist_k4.yaml: Configuration: OrthoHist-LoRA k=4

```
seed: 42
output_dir: ./outputs/ortho_hist_k4
trace_root: ./TRACE
model_name: Qwen/Qwen2.5-3B-Instruct
use_fast_tokenizer: false
padding_side: left
max_length: 1024

per_device_batch_size: 2
grad_accum_steps: 4
num_epochs: 3
learning_rate: 2.0e-4
weight_decay: 0.0
warmup_ratio: 0.03
max_grad_norm: 1.0
bf16: true
compile_model: false

log_every: 10
eval_every_epoch: false
save_every_task: true
debug_eval: false

method: ortho_hist

rank: 16
alpha: 32
dropout: 0.0
mt_rank: 4

target_modules:
  - q_proj
  - k_proj
  - v_proj
```

```
    - o_proj
    - gate_proj
    - up_proj
    - down_proj

train_tasks:
    - C-STANCE
    - FOMC
    - ScienceQA
    - NumGLUE-cm
    - NumGLUE-ds

eval_tasks:
    - C-STANCE
    - FOMC
    - ScienceQA
    - NumGLUE-cm
    - NumGLUE-ds
```

A.7 Configuration: OrthoHist-LoRA k=8

Listing A.7: configs/ortho_hist_k8.yaml: Configuration: OrthoHist-LoRA k=8

```
seed: 42
output_dir: ./outputs/ortho_hist_k8
trace_root: ./TRACE
model_name: Qwen/Qwen2.5-3B-Instruct
use_fast_tokenizer: false
padding_side: left
max_length: 1024
per_device_batch_size: 2
grad_accum_steps: 4
num_epochs: 3
learning_rate: 0.0002
weight_decay: 0.0
warmup_ratio: 0.03
max_grad_norm: 1.0
bf16: true
compile_model: false
log_every: 10
eval_every_epoch: false
save_every_task: true
method: ortho_hist
rank: 16
alpha: 32
dropout: 0.0
mt_rank: 8
debug_eval: false
target_modules:
- q_proj
- k_proj
- v_proj
- o_proj
- gate_proj
- up_proj
- down_proj
train_tasks:
- C-STANCE
```

```
- FOMC
- ScienceQA
- NumGLUE-cm
- NumGLUE-ds
eval_tasks:
- C-STANCE
- FOMC
- ScienceQA
- NumGLUE-cm
- NumGLUE-ds
```

A.8 Configuration: OrthoHist-LoRA k=16

Listing A.8: configs/ortho_hist_k16.yaml: Configuration: OrthoHist-LoRA k=16

```
seed: 42
output_dir: ./outputs/ortho_hist_k16
trace_root: ./TRACE
model_name: Qwen/Qwen2.5-3B-Instruct
use_fast_tokenizer: false
padding_side: left
max_length: 1024

per_device_batch_size: 2
grad_accum_steps: 4
num_epochs: 3
learning_rate: 2.0e-4
weight_decay: 0.0
warmup_ratio: 0.03
max_grad_norm: 1.0
bf16: true
compile_model: false

log_every: 10
eval_every_epoch: false
save_every_task: true
debug_eval: false

method: ortho_hist

rank: 16
alpha: 32
dropout: 0.0
mt_rank: 16

target_modules:
  - q_proj
  - k_proj
  - v_proj
  - o_proj
  - gate_proj
  - up_proj
  - down_proj

train_tasks:
  - C-STANCE
  - FOMC
  - ScienceQA
```

```
    - NumGLUE-cm
    - NumGLUE-ds

eval_tasks:
    - C-STANCE
    - FOMC
    - ScienceQA
    - NumGLUE-cm
    - NumGLUE-ds
```

A.9 Configuration Loader

Listing A.9: src/config.py: Configuration Loader

```python
from dataclasses import dataclass, field
from pathlib import Path
from typing import List

import yaml

# !
@dataclass
class TrainConfig:
    # paths and model
    seed: int = 42
    output_dir: str = "./outputs/run"
    trace_root: str = "./TRACE"
    model_name: str = "Qwen/Qwen2.5-3B-Instruct"
    use_fast_tokenizer: bool = False
    padding_side: str = "left"
    max_length: int = 1024

    # training
    per_device_batch_size: int = 2
    grad_accum_steps: int = 4
    num_epochs: int = 3
    learning_rate: float = 2.0e-4
    weight_decay: float = 0.0
    warmup_ratio: float = 0.03
    max_grad_norm: float = 1.0
    bf16: bool = True
    compile_model: bool = False

    # logging and checkpointing
    log_every: int = 10
    eval_every_epoch: bool = False
    save_every_task: bool = True
    debug_eval: bool = False

    # method: "ortho_hist" | "sequential_lora"
    method: str = "ortho_hist"

    # LoRA
    rank: int = 16
    alpha: int = 32
    dropout: float = 0.0
    target_modules: List[str] = field(default_factory=lambda: [
        "q_proj", "k_proj", "v_proj", "o_proj",
```

```
        "gate_proj", "up_proj", "down_proj",
    ])

    # OrthoHist-specific
    mt_rank: int = 8

    # task schedule
    train_tasks: List[str] = field(default_factory=lambda: [
        "C-STANCE", "FOMC", "ScienceQA", "NumGLUE-cm", "NumGLUE-ds",
    ])
    eval_tasks: List[str] = field(default_factory=lambda: [
        "C-STANCE", "FOMC", "ScienceQA", "NumGLUE-cm", "NumGLUE-ds",
    ])


def load_config(path):
    with open(path, "r", encoding="utf-8") as f:
        raw = yaml.safe_load(f) or {}

    valid_keys = set(TrainConfig.__dataclass_fields__.keys())
    unknown = sorted(set(raw.keys()) - valid_keys)
    if unknown:
        raise ValueError(f"Unknown config keys in {path}: {unknown}")

    cfg = TrainConfig(**raw)
    Path(cfg.output_dir).mkdir(parents=True, exist_ok=True)
    return cfg
```

A.10 TRACE Data Loader

Listing A.10: src/data_trace.py: TRACE Data Loader

```
from __future__ import annotations

import json
from dataclasses import dataclass
from pathlib import Path

import torch
from torch.utils.data import DataLoader, Dataset


TRACE_TASKS = [
    "C-STANCE",
    "FOMC",
    "ScienceQA",
    "NumGLUE-cm",
    "NumGLUE-ds",
]

TASK_METRICS = {
    "C-STANCE": "accuracy",
    "FOMC": "accuracy",
    "ScienceQA": "accuracy",
    "NumGLUE-cm": "exact_match",
    "NumGLUE-ds": "exact_match",
}

```

```python

@dataclass
class EncodedExample:
    input_ids: torch.Tensor
    labels: torch.Tensor
    prompt_text: str
    answer_text: str
    task_name: str


class TraceSFTDataset(Dataset):
    def __init__(
        self,
        path,
        tokenizer,
        max_length=1024,
        task_name=None,
        split="train",
        max_eval_samples=100,
    ):
        path = Path(path)
        if not path.exists():
            raise FileNotFoundError(f"TRACE split file not found: {path}")

        with open(path, "r", encoding="utf-8") as f:
            self.data = json.load(f)

        self.split = split
        self.tokenizer = tokenizer
        self.max_length = int(max_length)
        self.task_name = task_name or path.parent.name

        if split in {"eval", "test"} and max_eval_samples is not None:
            self.data = self.data[: int(max_eval_samples)]

    def __len__(self):
        return len(self.data)

    def _build_prompt_text(self, prompt):
        return self.tokenizer.apply_chat_template(
            [{"role": "user", "content": prompt}],
            tokenize=False,
            add_generation_prompt=True,
        )

    def _build_full_text(self, prompt, answer):
        return self.tokenizer.apply_chat_template(
            [
                {"role": "user", "content": prompt},
                {"role": "assistant", "content": answer},
            ],
            tokenize=False,
            add_generation_prompt=False,
        )

    def _encode_with_mask(self, prompt, answer):
        prompt_text = self._build_prompt_text(prompt)
        full_text = self._build_full_text(prompt, answer)

        prompt_ids = self.tokenizer(prompt_text, add_special_tokens=False).input_ids
        full_ids = self.tokenizer(full_text, add_special_tokens=False).input_ids
```

```
        answer_ids = full_ids[len(prompt_ids):]

        if len(full_ids) <= self.max_length:
            kept_prompt_ids = prompt_ids
            kept_answer_ids = answer_ids
        else:
            min_answer_tokens = max(1, min(len(answer_ids), 128))
            prompt_budget = max(0, self.max_length - min_answer_tokens)

            kept_prompt_ids = prompt_ids[-prompt_budget:] if prompt_budget > 0 else []
            answer_budget = self.max_length - len(kept_prompt_ids)
            kept_answer_ids = answer_ids[:answer_budget]

            if kept_answer_ids and len(answer_ids) > len(kept_answer_ids):
                eos_ids = self.tokenizer(
                    self.tokenizer.eos_token or "<|im_end|>",
                    add_special_tokens=False,
                ).input_ids
                if eos_ids:
                    kept_answer_ids[-1] = eos_ids[-1]

        kept_ids = kept_prompt_ids + kept_answer_ids
        labels = [-100] * len(kept_prompt_ids) + kept_answer_ids.copy()

        return EncodedExample(
            input_ids=torch.tensor(kept_ids, dtype=torch.long),
            labels=torch.tensor(labels, dtype=torch.long),
            prompt_text=prompt,
            answer_text=answer,
            task_name=self.task_name,
        )

    def __getitem__(self, idx):
        sample = self.data[idx]

        if "prompt" not in sample or "answer" not in sample:
            raise KeyError(
                f"Each TRACE sample must contain 'prompt' and 'answer'. "
                f"Bad sample in {self.task_name}/{self.split}: {sample}"
            )

        if self.split in {"eval", "test"}:
            return {
                "prompt_text": sample["prompt"],
                "answer_text": sample["answer"],
                "task_name": self.task_name,
            }

        encoded = self._encode_with_mask(sample["prompt"], sample["answer"])
        return {
            "input_ids": encoded.input_ids,
            "labels": encoded.labels,
            "prompt_text": encoded.prompt_text,
            "answer_text": encoded.answer_text,
            "task_name": encoded.task_name,
        }


def left_pad_stack(seqs, pad_value):
    max_len = max(x.size(0) for x in seqs)
    out = []
```

```
    for x in seqs:
        pad_len = max_len - x.size(0)
        if pad_len > 0:
            pad = torch.full((pad_len,), pad_value, dtype=x.dtype)
            out.append(torch.cat([pad, x], dim=0))
        else:
            out.append(x)
    return torch.stack(out, dim=0)


def make_trace_collator(tokenizer):
    pad_token_id = tokenizer.pad_token_id

    def collate(features):
        if "input_ids" not in features[0]:
            return {
                "prompt_text": [f["prompt_text"] for f in features],
                "answer_text": [f["answer_text"] for f in features],
                "task_name": [f["task_name"] for f in features],
            }

        input_ids = left_pad_stack([f["input_ids"] for f in features], pad_token_id)
        labels = left_pad_stack([f["labels"] for f in features], -100)
        attention_mask = (input_ids != pad_token_id).long()

        return {
            "input_ids": input_ids,
            "labels": labels,
            "attention_mask": attention_mask,
            "prompt_text": [f["prompt_text"] for f in features],
            "answer_text": [f["answer_text"] for f in features],
            "task_name": [f["task_name"] for f in features],
        }

    return collate


def build_trace_loader(
    trace_root,
    task_name,
    split,
    tokenizer,
    batch_size,
    max_length,
    shuffle=False,
    num_workers=0,
):
    path = Path(trace_root) / task_name / f"{split}.json"

    dataset = TraceSFTDataset(
        path,
        tokenizer=tokenizer,
        max_length=max_length,
        task_name=task_name,
        split=split,
    )

    return DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
```

```
        num_workers=num_workers,
        collate_fn=make_trace_collator(tokenizer),
        pin_memory=torch.cuda.is_available(),
    )


def build_task_loaders(
    trace_root,
    tasks,
    tokenizer,
    batch_size,
    max_length,
):
    loaders = {}

    for task in tasks:
        loaders[task] = {
            "train": build_trace_loader(
                trace_root,
                task,
                "train",
                tokenizer,
                batch_size,
                max_length,
                shuffle=True,
            ),
            "eval": build_trace_loader(
                trace_root,
                task,
                "eval",
                tokenizer,
                20,
                max_length,
                shuffle=False,
            ),
            "test": build_trace_loader(
                trace_root,
                task,
                "test",
                tokenizer,
                20,
                max_length,
                shuffle=False,
            ),
        }

    return loaders
```

A.11 TRACE Evaluation Utilities

Listing A.11: src/eval_trace.py: TRACE Evaluation Utilities

```
from __future__ import annotations

import re
from dataclasses import dataclass
from decimal import Decimal, InvalidOperation

```

```python
from .data_trace import TASK_METRICS

def normalize_text(text):
    if text is None:
        return ""
    return " ".join(str(text).strip().split()).lower()


def parse_explicit_choice(text, valid_choices=("A", "B", "C")):
    if text is None:
        return None

    raw = str(text).strip()
    if not raw:
        return None

    choices = "".join(valid_choices)
    choices_lower = choices.lower()

    m = re.search(
        rf"(?:the\s+correct\s+answer|correct\s+answer|the\s+answer|answer|||choice|option|label|result||
 stance)"
        rf"\s*(?:is|||=|:|)?\s*"
        rf"([{choices}{choices_lower}])"
        rf"(?:\s|$|[\.\)\],:;])",
        raw,
        flags=re.IGNORECASE,
    )
    if m:
        return m.group(1).upper()

    m = re.match(
        rf"^\s*[\(\[]?\s*([{choices}])\s*[\)\]]?"
        rf"(?:\s|$|[\.\,:;])",
        raw,
    )
    if m:
        return m.group(1).upper()

    m = re.match(
        rf"^\s*[\(\[]?\s*([{choices_lower}])\s*[\)\]]?"
        rf"(?:$|[\.\,:;])",
        raw,
    )
    if m:
        return m.group(1).upper()

    m = re.search(
        rf"(?:choose|select|selected|chooses|choosing|option|choice|)\s*"
        rf"([{choices}{choices_lower}])"
        rf"(?:\s|$|[\.\)\],:;])",
        raw,
        flags=re.IGNORECASE,
    )
    if m:
        return m.group(1).upper()

    return None


def parse_cstance_answer(text):
```

```
    label = parse_explicit_choice(text, valid_choices=("A", "B", "C"))
    if label is not None:
        return label

    if text is None:
        return None

    t = normalize_text(text)

    m = re.search(
        r"(?:|||answer|label|stance)"
        r"\s*(?:||=|:|)?\s*"
        r"(||)",
        t,
        flags=re.IGNORECASE,
    )
    if m:
        return {"": "A", "": "B", "": "C"}[m.group(1)]

    m = re.match(r"^\s*(||)(?:\s|$|||,|\.||!)", t)
    if m:
        return {"": "A", "": "B", "": "C"}[m.group(1)]

    found = set()

    if re.search(r"(||)", t):
        found.add("B")

    if re.search(r"(?<!)||", t):
        found.add("A")

    if re.search(r"(||)", t):
        found.add("C")

    if len(found) == 1:
        return next(iter(found))

    return None


def parse_fomc_answer(text):
    label = parse_explicit_choice(text, valid_choices=("A", "B", "C"))
    if label is not None:
        return label

    if text is None:
        return None

    t = normalize_text(text)
    found = set()

    if re.search(r"\bdovish\b", t, flags=re.IGNORECASE):
        found.add("A")

    if re.search(r"\bhawkish\b", t, flags=re.IGNORECASE):
        found.add("B")

    if re.search(r"\bneutral\b", t, flags=re.IGNORECASE):
        found.add("C")

    if len(found) == 1:
```

```
        return next(iter(found))

    return None


def parse_scienceqa_answer(text):
    return parse_explicit_choice(
        text,
        valid_choices=("A", "B", "C", "D", "E"),
    )


def parse_answer_for_task(task_name, text):
    if task_name == "C-STANCE":
        return parse_cstance_answer(text)

    if task_name == "FOMC":
        return parse_fomc_answer(text)

    if task_name == "ScienceQA":
        return parse_scienceqa_answer(text)

    return normalize_text(text)


MONTHS = {
    "january", "february", "march", "april", "may", "june",
    "july", "august", "september", "october", "november", "december",
}


def parse_decimal(text):
    if text is None:
        return None

    s = str(text).strip().replace(",", "")

    try:
        return Decimal(s)
    except InvalidOperation:
        return None


def extract_explicit_number(text):
    if text is None:
        return None

    raw = str(text).replace(",", "")

    m = re.search(
        r"(?:the\s+answer|answer|final\s+answer|result|)"
        r"\s*(?:is|||=|:|)?\s*"
        r"([-+]?\d+(?:\.\d+)?)",
        raw,
        flags=re.IGNORECASE,
    )

    if m:
        return parse_decimal(m.group(1))

    return None
```

```python


def extract_leading_number(text):
    if text is None:
        return None

    raw = str(text).strip().replace(",", "")

    m = re.match(
        r"^\s*([-+]?\d+(?:\.\d+)?)(?:\s|$|[%\.\,\;\:\)\]])",
        raw,
    )

    if m:
        return parse_decimal(m.group(1))

    return None


def extract_last_number(text):
    if text is None:
        return None

    raw = str(text).replace(",", "")
    nums = re.findall(r"[-+]?\d+(?:\.\d+)?", raw)

    if not nums:
        return None

    return parse_decimal(nums[-1])


def extract_number_answer(text):
    num = extract_explicit_number(text)
    if num is not None:
        return num

    num = extract_leading_number(text)
    if num is not None:
        return num

    return extract_last_number(text)


def month_pattern():
    return (
        r"(?:january|february|march|april|may|june|july|august|"
        r"september|october|november|december)"
    )


def extract_explicit_month(text):
    if text is None:
        return None

    raw = str(text).strip()

    m = re.search(
        rf"(?:the\s+answer|answer|final\s+answer|result|)"
        rf"\s*(?:is|||=|:|)?\s*"
        rf"({month_pattern()})"
```

```
        rf"(?:\s|$|[\.\)\],:;])",
        raw,
        flags=re.IGNORECASE,
    )

    if m:
        return normalize_text(m.group(1))

    return None


def extract_leading_month(text):
    if text is None:
        return None

    raw = str(text).strip()

    m = re.match(
        rf"^\s*({month_pattern()})(?:\s|$|[\.\)\],:;])",
        raw,
        flags=re.IGNORECASE,
    )

    if m:
        return normalize_text(m.group(1))

    return None


def extract_last_month(text):
    if text is None:
        return None

    t = normalize_text(text)
    hits = []

    for month in MONTHS:
        for m in re.finditer(rf"\b{month}\b", t):
            hits.append((m.start(), month))

    if not hits:
        return None

    hits.sort(key=lambda x: x[0])
    return hits[-1][1]


def extract_month_answer(text):
    month = extract_explicit_month(text)
    if month is not None:
        return month

    month = extract_leading_month(text)
    if month is not None:
        return month

    return extract_last_month(text)


def clean_final_text_answer(text):
    if text is None:
```

```
        return ""

    raw = str(text).strip()

    m = re.search(
        r"(?:the\s+answer|answer|final\s+answer|result|)"
        r"\s*(?:is|||=|:|)?\s*"
        r"([a-zA-Z]+)",
        raw,
        flags=re.IGNORECASE,
    )
    if m:
        return normalize_text(m.group(1)).strip(".,;:!?")

    return normalize_text(raw).strip(".,;:!?")


def exact_match_score(preds, refs):
    correct = 0
    total = len(preds)

    for pred, ref in zip(preds, refs):
        ref_num = parse_decimal(ref)

        if ref_num is not None:
            pred_num = extract_number_answer(pred)

            if pred_num is not None and pred_num == ref_num:
                correct += 1

        else:
            ref_month = extract_month_answer(ref)

            if ref_month is not None:
                pred_month = extract_month_answer(pred)

                if pred_month is not None and pred_month == ref_month:
                    correct += 1

            else:
                pred_clean = clean_final_text_answer(pred)
                ref_clean = clean_final_text_answer(ref)

                if pred_clean == ref_clean:
                    correct += 1

    return correct / max(1, total)


def accuracy_score(preds, refs, task_name):
    correct = 0
    total = len(preds)

    for pred, ref in zip(preds, refs):
        if task_name in {"C-STANCE", "FOMC", "ScienceQA"}:
            parsed_pred = parse_answer_for_task(task_name, pred)
            parsed_ref = parse_answer_for_task(task_name, ref)

            if (
                parsed_pred is not None
                and parsed_ref is not None
```

```
                and parsed_pred == parsed_ref
            ):
                correct += 1

        else:
            pred_clean = normalize_text(pred)
            ref_clean = normalize_text(ref)

            if pred_clean == ref_clean:
                correct += 1

    return correct / max(1, total)


@dataclass
class TaskEvalResult:
    task_name: str
    metric_name: str
    metric_value: float
    num_examples: int


def score_task(task_name, preds, refs, sources=None):
    metric_name = TASK_METRICS.get(task_name, "exact_match")

    if metric_name == "accuracy":
        value = accuracy_score(preds, refs, task_name)

    elif metric_name == "exact_match":
        value = exact_match_score(preds, refs)

    else:
        raise ValueError(f"Unsupported metric: {metric_name}")

    return TaskEvalResult(
        task_name=task_name,
        metric_name=metric_name,
        metric_value=value,
        num_examples=len(preds),
    )


def parse_for_debug(task_name, pred, ref):
    metric_name = TASK_METRICS.get(task_name, "exact_match")

    if metric_name == "accuracy":
        parsed_pred = parse_answer_for_task(task_name, pred)
        parsed_ref = parse_answer_for_task(task_name, ref)
        correct = (
            parsed_pred is not None
            and parsed_ref is not None
            and parsed_pred == parsed_ref
        )
        return parsed_pred, parsed_ref, correct

    if metric_name == "exact_match":
        ref_num = parse_decimal(ref)

        if ref_num is not None:
            parsed_pred = extract_number_answer(pred)
            parsed_ref = ref_num
```

```
            correct = (
                parsed_pred is not None
                and parsed_ref is not None
                and parsed_pred == parsed_ref
            )
            return parsed_pred, parsed_ref, correct

        ref_month = extract_month_answer(ref)

        if ref_month is not None:
            parsed_pred = extract_month_answer(pred)
            parsed_ref = ref_month
            correct = (
                parsed_pred is not None
                and parsed_ref is not None
                and parsed_pred == parsed_ref
            )
            return parsed_pred, parsed_ref, correct

        parsed_pred = clean_final_text_answer(pred)
        parsed_ref = clean_final_text_answer(ref)
        correct = (
            parsed_pred is not None
            and parsed_ref is not None
            and parsed_pred == parsed_ref
        )
        return parsed_pred, parsed_ref, correct

    return None, None, False


def debug_parse_examples(task_name, preds, refs, n=20):
    for i, (pred, ref) in enumerate(zip(preds[:n], refs[:n])):
        parsed_pred, parsed_ref, correct = parse_for_debug(task_name, pred, ref)

        print("=" * 80)
        print(f"idx: {i}")
        print(f"task: {task_name}")
        print(f"raw pred: {repr(pred)}")
        print(f"raw ref : {repr(ref)}")
        print(f"parsed pred: {parsed_pred}")
        print(f"parsed ref : {parsed_ref}")
        print(f"correct: {correct}")


def build_debug_rows(task_name, preds, refs, prompts):
    rows = []

    for prompt, pred, ref in zip(prompts, preds, refs):
        parsed_pred, parsed_ref, correct = parse_for_debug(task_name, pred, ref)

        rows.append({
            "task": task_name,
            "prompt": prompt,
            "ref": ref,
            "pred": pred,
            "parsed_ref": str(parsed_ref) if parsed_ref is not None else None,
            "parsed_pred": str(parsed_pred) if parsed_pred is not None else None,
            "correct": correct,
        })

```

```
    return rows


def average_accuracy(results):
    return sum(results.values()) / len(results) if results else 0.0


def backward_transfer(score_matrix, trained_tasks):
    if len(trained_tasks) <= 1:
        return 0.0

    current = trained_tasks[-1]
    current_row = score_matrix.get(current, {})

    diffs = []
    for task in trained_tasks[:-1]:
        if task in current_row and task in score_matrix and task in score_matrix[task]:
            diffs.append(current_row[task] - score_matrix[task][task])

    return sum(diffs) / len(diffs) if diffs else 0.0


def forward_transfer(zero_shot_scores, score_matrix, trained_tasks):
    # If zero-shot scores are not explicitly computed, FWT is undefined in this
    # experiment code path. Return 0.0 to keep the existing table format stable.
    if len(trained_tasks) <= 1 or not zero_shot_scores:
        return 0.0

    total, count = 0.0, 0

    for i, task in enumerate(trained_tasks[1:], 1):
        prev = trained_tasks[i - 1]
        prev_row = score_matrix.get(prev, {})

        if task in prev_row and task in zero_shot_scores:
            total += prev_row[task] - zero_shot_scores[task]
            count += 1

    return total / count if count else 0.0
```

A.12 Training Loops

Listing A.12: src/trainers.py: Training Loops

```
import json
import math
import random
from pathlib import Path

import numpy as np
import torch
from torch.optim import AdamW
from tqdm import tqdm
from transformers import get_linear_schedule_with_warmup

from .analysis_utils import collect_reuse_matrix, save_reuse_heatmap, save_score_matrix
from .checkpointing import load_checkpoint, save_checkpoint
from .data_trace import build_task_loaders
```

```
from .eval_trace import (
    average_accuracy,
    backward_transfer,
    build_debug_rows,
    debug_parse_examples,
    forward_transfer,
    score_task,
)


def set_seed(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)


def _make_optimizer_and_scheduler(params, cfg, n_steps):
    opt = AdamW(params, lr=cfg.learning_rate, weight_decay=cfg.weight_decay)
    warmup = int(cfg.warmup_ratio * n_steps)
    sch = get_linear_schedule_with_warmup(opt, warmup, n_steps)
    return opt, sch


def _find_last_done_task(output_dir, n_tasks):
    """Return the last consecutively completed task id."""
    last_done = 0
    ckpt_root = Path(output_dir) / "checkpoints"

    for tid in range(1, n_tasks + 1):
        ckpt_dir = ckpt_root / f"task_{tid:02d}"
        done_flag = ckpt_dir / "DONE"
        state_file = ckpt_dir / "model_state.pt"

        if done_flag.exists() and state_file.exists():
            last_done = tid
        else:
            break

    return last_done


class JsonLogger:
    def __init__(self, output_dir):
        p = Path(output_dir) / "logs"
        p.mkdir(parents=True, exist_ok=True)
        self.path = p / "events.jsonl"
        self.path.touch(exist_ok=True)

    def log(self, payload):
        self.path.parent.mkdir(parents=True, exist_ok=True)
        with open(self.path, "a", encoding="utf-8") as f:
            f.write(json.dumps(payload, ensure_ascii=False) + "\n")


class ContinualTrainer:
    def __init__(self, cfg, tokenizer, model, manager, device):
        self.cfg = cfg
        self.tok = tokenizer
        self.model = model
        self.manager = manager
```

```
        self.device = device
        self.logger = JsonLogger(cfg.output_dir)

        self.loaders = build_task_loaders(
            trace_root=cfg.trace_root,
            tasks=cfg.train_tasks,
            tokenizer=tokenizer,
            batch_size=cfg.per_device_batch_size,
            max_length=cfg.max_length,
        )

        self.score_matrix = {}
        self.zero_shot_scores = {}
        self.reuse_snapshots = []
        self.history = []

    def _save_history(self):
        out = Path(self.cfg.output_dir)
        out.mkdir(parents=True, exist_ok=True)
        with open(out / "history.json", "w", encoding="utf-8") as f:
            json.dump(self.history, f, indent=2, ensure_ascii=False)

    def _save_score_matrix(self):
        save_score_matrix(
            self.score_matrix,
            self.cfg.train_tasks,
            self.cfg.output_dir,
        )

        out = Path(self.cfg.output_dir)
        out.mkdir(parents=True, exist_ok=True)
        with open(out / "score_matrix_raw.json", "w", encoding="utf-8") as f:
            json.dump(self.score_matrix, f, indent=2, ensure_ascii=False)

    def _save_reuse_snapshots(self):
        out = Path(self.cfg.output_dir)
        out.mkdir(parents=True, exist_ok=True)
        with open(out / "reuse_snapshots.json", "w", encoding="utf-8") as f:
            json.dump(self.reuse_snapshots, f, indent=2, ensure_ascii=False)

    def _restore_history_and_scores(self):
        out = Path(self.cfg.output_dir)

        hp = out / "history.json"
        if hp.exists():
            with open(hp, encoding="utf-8") as f:
                self.history = json.load(f)

        smp = out / "score_matrix_raw.json"
        if smp.exists():
            with open(smp, encoding="utf-8") as f:
                self.score_matrix = json.load(f)
            print(f"[Resume] restored score_matrix from {smp}")
        else:
            print("[Resume WARNING] score_matrix_raw.json not found; rebuilding from history.")
            self.score_matrix = {}
            for entry in self.history:
                key = entry.get("task_name") or entry.get("task")
                if key is not None:
                    self.score_matrix[key] = dict(entry.get("seen_scores", {}))

```

```
    def _restore_reuse_snapshots(self):
        snap_path = Path(self.cfg.output_dir) / "reuse_snapshots.json"
        if snap_path.exists():
            with open(snap_path, encoding="utf-8") as f:
                self.reuse_snapshots = json.load(f)
            print(f"[Resume] restored {len(self.reuse_snapshots)} reuse snapshots")

    def _preallocate_orthohist_pool_for_resume(self, n_committed_tasks):
        """Create basis_pool slots before loading an OrthoHist checkpoint."""
        if n_committed_tasks <= 0:
            return

        for layer in self.manager.layers.values():
            layer.basis_pool = torch.nn.ParameterList()
            device = layer.lora_B.device
            dtype = layer.lora_B.dtype

            for _ in range(n_committed_tasks):
                basis = torch.empty(
                    layer.out_features,
                    layer.rank,
                    device=device,
                    dtype=dtype,
                )
                coeff = torch.empty(
                    layer.rank,
                    layer.in_features,
                    device=device,
                    dtype=dtype,
                )
                layer.basis_pool.append(torch.nn.Parameter(basis, requires_grad=False))
                layer.basis_pool.append(torch.nn.Parameter(coeff, requires_grad=False))

            layer._n_pool = n_committed_tasks

    def _mark_done(self, task_id):
        done_flag = (
            Path(self.cfg.output_dir)
            / "checkpoints"
            / f"task_{task_id:02d}"
            / "DONE"
        )
        done_flag.parent.mkdir(parents=True, exist_ok=True)
        done_flag.touch()

    def _train_one_epoch(self, task_name, optimizer, scheduler, epoch):
        self.model.train()
        loader = self.loaders[task_name]["train"]
        total_loss = 0.0
        batch_cnt = 0
        optimizer.zero_grad(set_to_none=True)

        pbar = tqdm(loader, desc=f"[{task_name}] epoch {epoch}", leave=False)
        for step, batch in enumerate(pbar, 1):
            batch = {
                k: v.to(self.device) if isinstance(v, torch.Tensor) else v
                for k, v in batch.items()
            }

            out = self.model(
                input_ids=batch["input_ids"],
```

```
                attention_mask=batch["attention_mask"],
                labels=batch["labels"],
            )
            loss = out.loss / self.cfg.grad_accum_steps
            loss.backward()

            if step % self.cfg.grad_accum_steps == 0:
                torch.nn.utils.clip_grad_norm_(
                    self.manager.trainable_parameters(), self.cfg.max_grad_norm
                )
                optimizer.step()
                scheduler.step()
                optimizer.zero_grad(set_to_none=True)

            total_loss += float(out.loss.item())
            batch_cnt += 1
            if batch_cnt % self.cfg.log_every == 0:
                pbar.set_postfix(loss=f"{total_loss / batch_cnt:.4f}")

        if len(loader) % self.cfg.grad_accum_steps != 0:
            torch.nn.utils.clip_grad_norm_(
                self.manager.trainable_parameters(), self.cfg.max_grad_norm
            )
            optimizer.step()
            scheduler.step()
            optimizer.zero_grad(set_to_none=True)

        return total_loss / max(1, batch_cnt)

    @torch.no_grad()
    def _generate(self, task_name, split="test"):
        self.model.eval()
        loader = self.loaders[task_name][split]
        preds, refs, prompts = [], [], []

        for batch in tqdm(loader, desc=f"eval {task_name}/{split}", leave=False):
            prompt_texts = [
                self.tok.apply_chat_template(
                    [{"role": "user", "content": p}],
                    tokenize=False,
                    add_generation_prompt=True,
                )
                for p in batch["prompt_text"]
            ]

            enc = self.tok(
                prompt_texts,
                return_tensors="pt",
                padding=True,
                truncation=True,
                max_length=self.cfg.max_length,
                add_special_tokens=False,
            ).to(self.device)

            if task_name in {"C-STANCE", "FOMC"}:
                max_new_tokens = 8
            elif task_name == "ScienceQA":
                max_new_tokens = 64
            else:
                max_new_tokens = 128

```

```
            out = self.model.generate(
                input_ids=enc["input_ids"],
                attention_mask=enc["attention_mask"],
                max_new_tokens=max_new_tokens,
                eos_token_id=self.tok.eos_token_id,
                pad_token_id=self.tok.pad_token_id,
                do_sample=False,
            )

            input_len = enc["input_ids"].shape[1]
            for i in range(out.size(0)):
                pred = self.tok.decode(out[i][input_len:], skip_special_tokens=True)
                preds.append(pred.strip())

            refs.extend(batch["answer_text"])
            prompts.extend(batch["prompt_text"])

        if getattr(self.cfg, "debug_eval", False):
            debug_parse_examples(task_name, preds, refs, n=10)

        return {"preds": preds, "refs": refs, "prompts": prompts}

    def _save_raw_predictions(self, row_key, task_name, split, gen_results):
        debug_dir = Path(self.cfg.output_dir) / "debug_predictions"
        debug_dir.mkdir(parents=True, exist_ok=True)

        safe_row = str(row_key).replace("/", "_")
        safe_task = str(task_name).replace("/", "_")
        out_path = debug_dir / f"{safe_row}__{safe_task}__{split}.jsonl"

        rows = build_debug_rows(
            task_name=task_name,
            preds=gen_results["preds"],
            refs=gen_results["refs"],
            prompts=gen_results["prompts"],
        )

        with open(out_path, "w", encoding="utf-8") as f:
            for row in rows:
                f.write(json.dumps(row, ensure_ascii=False) + "\n")

    def _eval_tasks(self, task_list, row_key, split="test"):
        self.score_matrix.setdefault(row_key, {})
        task_scores = {}

        for task in task_list:
            gen_results = self._generate(task, split=split)
            self._save_raw_predictions(
                row_key=row_key,
                task_name=task,
                split=split,
                gen_results=gen_results,
            )

            res = score_task(
                task,
                gen_results["preds"],
                gen_results["refs"],
                gen_results["prompts"],
            )

```

```
            task_scores[task] = res.metric_value
            self.score_matrix[row_key][task] = res.metric_value

            self.logger.log({
                "event": "task_eval",
                "row": row_key,
                "task": task,
                "split": split,
                "metric": res.metric_name,
                "value": res.metric_value,
                "num_examples": res.num_examples,
            })

        return task_scores

    def train_one_task(self, task_name, task_id, seen):
        self.manager.prepare_for_task(task_id)

        n_steps = max(
            1,
            math.ceil(
                len(self.loaders[task_name]["train"]) * self.cfg.num_epochs
                / self.cfg.grad_accum_steps
            ),
        )
        opt, sch = _make_optimizer_and_scheduler(
            self.manager.trainable_parameters(), self.cfg, n_steps
        )

        self.logger.log({
            "event": "task_start",
            "task": task_name,
            "task_id": task_id,
            "trainable_params": sum(p.numel() for p in self.manager.trainable_parameters()),
        })

        for epoch in range(1, self.cfg.num_epochs + 1):
            loss = self._train_one_epoch(task_name, opt, sch, epoch)
            self.logger.log({
                "event": "epoch_end",
                "task": task_name,
                "epoch": epoch,
                "loss": loss,
            })

        print(f"[{task_name}] training done, running eval...")
        self._eval_tasks(self.cfg.train_tasks, row_key=task_name, split="test")

        if hasattr(self.manager, "collect_reuse_analysis"):
            snapshot = self.manager.collect_reuse_analysis(task_name)
            self.reuse_snapshots.append(snapshot)
            self._save_reuse_snapshots()

        self.manager.commit_task()

        scores = {t: self.score_matrix[task_name][t] for t in seen}
        aa = average_accuracy(scores)
        bwt = backward_transfer(self.score_matrix, seen)
        fwt = forward_transfer(self.zero_shot_scores, self.score_matrix, seen)

        self.history.append({
```

```
            "task_id": task_id,
            "task_name": task_name,
            "seen_scores": scores,
            "average_accuracy": aa,
            "bwt": bwt,
            "fwt": fwt,
        })

        self._save_history()
        self._save_score_matrix()

        self.logger.log({
            "event": "task_end",
            "task": task_name,
            "aa": aa,
            "bwt": bwt,
            "fwt": fwt,
        })
        print(f"[{task_name}] AA={aa:.4f}  BWT={bwt:.4f}  FWT={fwt:.4f}")

        if self.cfg.save_every_task:
            save_checkpoint(
                self.cfg.output_dir,
                task_id,
                self.model,
                extra={
                    "task": task_name,
                    "aa": aa,
                    "bwt": bwt,
                    "fwt": fwt,
                },
            )
            self._mark_done(task_id)

    def train_all_tasks(self):
        seen = []
        last_done = _find_last_done_task(self.cfg.output_dir, len(self.cfg.train_tasks))

        if last_done > 0:
            print(
                f"[Resume] Loading checkpoint from task {last_done} "
                f"({self.cfg.train_tasks[last_done - 1]})"
            )
            self._preallocate_orthohist_pool_for_resume(last_done)
            load_checkpoint(self.cfg.output_dir, last_done, self.model)

            for layer in self.manager.layers.values():
                layer._n_pool = last_done

            self._restore_history_and_scores()
            self._restore_reuse_snapshots()

        for task_id, task in enumerate(self.cfg.train_tasks, 1):
            seen.append(task)
            if task_id <= last_done:
                print(f"[Resume] skipping {task} (already done)")
                continue
            self.train_one_task(task, task_id, seen)

        self._finalise()
        return {"history": self.history, "score_matrix": self.score_matrix}
```

```
442
443     def _finalise(self):
444         self._save_history()
445         self._save_score_matrix()
446
447         if self.reuse_snapshots:
448             matrix = collect_reuse_matrix(self.reuse_snapshots, self.cfg.train_tasks)
449             save_reuse_heatmap(matrix, self.cfg.train_tasks, self.cfg.output_dir)
450
451
452 class SequentialLoRATrainer:
453     def __init__(self, cfg, tokenizer, model, device):
454         self.cfg = cfg
455         self.tok = tokenizer
456         self.model = model
457         self.device = device
458         self.loaders = build_task_loaders(
459             trace_root=cfg.trace_root,
460             tasks=cfg.train_tasks,
461             tokenizer=tokenizer,
462             batch_size=cfg.per_device_batch_size,
463             max_length=cfg.max_length,
464         )
465         self.score_matrix = {}
466         self.zero_shot_scores = {}
467         self.history = []
468         self.logger = JsonLogger(cfg.output_dir)
469
470         self._set_lora_trainable_only()
471
472     def _set_lora_trainable_only(self):
473         for _, p in self.model.named_parameters():
474             p.requires_grad_(False)
475
476         trainable = []
477         for name, p in self.model.named_parameters():
478             if "lora_A" in name or "lora_B" in name:
479                 p.requires_grad_(True)
480                 trainable.append(p)
481
482         if not trainable:
483             raise RuntimeError("No LoRA parameters found in SequentialLoRATrainer.")
484
485     def _trainable_parameters(self):
486         return [p for p in self.model.parameters() if p.requires_grad]
487
488     def _save_history(self):
489         out = Path(self.cfg.output_dir)
490         out.mkdir(parents=True, exist_ok=True)
491         with open(out / "history.json", "w", encoding="utf-8") as f:
492             json.dump(self.history, f, indent=2, ensure_ascii=False)
493
494     def _save_score_matrix(self):
495         save_score_matrix(
496             self.score_matrix,
497             self.cfg.train_tasks,
498             self.cfg.output_dir,
499         )
500
501         out = Path(self.cfg.output_dir)
502         out.mkdir(parents=True, exist_ok=True)
```

```
        with open(out / "score_matrix_raw.json", "w", encoding="utf-8") as f:
            json.dump(self.score_matrix, f, indent=2, ensure_ascii=False)

    def _restore_history_and_scores(self):
        out = Path(self.cfg.output_dir)

        hp = out / "history.json"
        if hp.exists():
            with open(hp, encoding="utf-8") as f:
                self.history = json.load(f)

        smp = out / "score_matrix_raw.json"
        if smp.exists():
            with open(smp, encoding="utf-8") as f:
                self.score_matrix = json.load(f)
            print(f"[Resume] restored score_matrix from {smp}")
        else:
            print("[Resume WARNING] score_matrix_raw.json not found; rebuilding from history.")
            self.score_matrix = {}
            for entry in self.history:
                key = entry.get("task_name") or entry.get("task")
                if key is not None:
                    self.score_matrix[key] = dict(entry.get("seen_scores", {}))

    def _mark_done(self, task_id):
        done_flag = (
            Path(self.cfg.output_dir)
            / "checkpoints"
            / f"task_{task_id:02d}"
            / "DONE"
        )
        done_flag.parent.mkdir(parents=True, exist_ok=True)
        done_flag.touch()

    @torch.no_grad()
    def _generate(self, task_name, split="test"):
        self.model.eval()
        loader = self.loaders[task_name][split]
        preds, refs, prompts = [], [], []

        for batch in tqdm(loader, desc=f"eval {task_name}/{split}", leave=False):
            prompt_texts = [
                self.tok.apply_chat_template(
                    [{"role": "user", "content": p}],
                    tokenize=False,
                    add_generation_prompt=True,
                )
                for p in batch["prompt_text"]
            ]

            enc = self.tok(
                prompt_texts,
                return_tensors="pt",
                padding=True,
                truncation=True,
                max_length=self.cfg.max_length,
                add_special_tokens=False,
            ).to(self.device)

            if task_name in {"C-STANCE", "FOMC"}:
                max_new_tokens = 8
```

```
            elif task_name == "ScienceQA":
                max_new_tokens = 64
            else:
                max_new_tokens = 128

            out = self.model.generate(
                input_ids=enc["input_ids"],
                attention_mask=enc["attention_mask"],
                max_new_tokens=max_new_tokens,
                eos_token_id=self.tok.eos_token_id,
                pad_token_id=self.tok.pad_token_id,
                do_sample=False,
            )

            input_len = enc["input_ids"].shape[1]
            for i in range(out.size(0)):
                pred = self.tok.decode(out[i][input_len:], skip_special_tokens=True)
                preds.append(pred.strip())

            refs.extend(batch["answer_text"])
            prompts.extend(batch["prompt_text"])

        if getattr(self.cfg, "debug_eval", False):
            debug_parse_examples(task_name, preds, refs, n=10)

        return {"preds": preds, "refs": refs, "prompts": prompts}

    def _save_raw_predictions(self, row_key, task_name, split, gen_results):
        debug_dir = Path(self.cfg.output_dir) / "debug_predictions"
        debug_dir.mkdir(parents=True, exist_ok=True)

        safe_row = str(row_key).replace("/", "_")
        safe_task = str(task_name).replace("/", "_")
        out_path = debug_dir / f"{safe_row}__{safe_task}__{split}.jsonl"

        rows = build_debug_rows(
            task_name=task_name,
            preds=gen_results["preds"],
            refs=gen_results["refs"],
            prompts=gen_results["prompts"],
        )

        with open(out_path, "w", encoding="utf-8") as f:
            for row in rows:
                f.write(json.dumps(row, ensure_ascii=False) + "\n")

    def _eval_tasks(self, task_list, row_key, split="test"):
        self.score_matrix.setdefault(row_key, {})
        task_scores = {}

        for task in task_list:
            gen_results = self._generate(task, split=split)
            self._save_raw_predictions(
                row_key=row_key,
                task_name=task,
                split=split,
                gen_results=gen_results,
            )

            res = score_task(
                task,
```

```
                gen_results["preds"],
                gen_results["refs"],
                gen_results["prompts"],
            )

            task_scores[task] = res.metric_value
            self.score_matrix[row_key][task] = res.metric_value

            self.logger.log({
                "event": "task_eval",
                "row": row_key,
                "task": task,
                "split": split,
                "metric": res.metric_name,
                "value": res.metric_value,
                "num_examples": res.num_examples,
            })

        return task_scores

    def _train_one_task(self, task_name, task_id):
        self._set_lora_trainable_only()
        trainable = self._trainable_parameters()
        loader = self.loaders[task_name]["train"]

        n_steps = max(
            1,
            math.ceil(
                len(loader) * self.cfg.num_epochs
                / self.cfg.grad_accum_steps
            ),
        )
        opt, sch = _make_optimizer_and_scheduler(trainable, self.cfg, n_steps)
        self.model.train()
        opt.zero_grad(set_to_none=True)

        self.logger.log({
            "event": "task_start",
            "task": task_name,
            "task_id": task_id,
            "trainable_params": sum(p.numel() for p in trainable),
        })

        for epoch in range(1, self.cfg.num_epochs + 1):
            total_loss = 0.0
            batch_cnt = 0
            pbar = tqdm(loader, desc=f"seq_lora [{task_name}] ep{epoch}", leave=False)

            for step, batch in enumerate(pbar, 1):
                batch = {
                    k: v.to(self.device) if isinstance(v, torch.Tensor) else v
                    for k, v in batch.items()
                }
                out = self.model(
                    input_ids=batch["input_ids"],
                    attention_mask=batch["attention_mask"],
                    labels=batch["labels"],
                )
                (out.loss / self.cfg.grad_accum_steps).backward()

                if step % self.cfg.grad_accum_steps == 0:
```

```
                    torch.nn.utils.clip_grad_norm_(trainable, self.cfg.max_grad_norm)
                    opt.step()
                    sch.step()
                    opt.zero_grad(set_to_none=True)

                total_loss += float(out.loss.item())
                batch_cnt += 1
                if batch_cnt % self.cfg.log_every == 0:
                    pbar.set_postfix(loss=f"{total_loss / batch_cnt:.4f}")

            if len(loader) % self.cfg.grad_accum_steps != 0:
                torch.nn.utils.clip_grad_norm_(trainable, self.cfg.max_grad_norm)
                opt.step()
                sch.step()
                opt.zero_grad(set_to_none=True)

            self.logger.log({
                "event": "epoch_end",
                "task": task_name,
                "epoch": epoch,
                "loss": total_loss / max(1, batch_cnt),
            })

    def train_all_tasks(self):
        seen = []
        last_done = _find_last_done_task(self.cfg.output_dir, len(self.cfg.train_tasks))

        if last_done > 0:
            print(
                f"[Resume] Loading Sequential LoRA checkpoint from task {last_done} "
                f"({self.cfg.train_tasks[last_done - 1]})"
            )
            load_checkpoint(self.cfg.output_dir, last_done, self.model)
            self.model.to(self.device)
            self._set_lora_trainable_only()
            self._restore_history_and_scores()

        for task_id, task in enumerate(self.cfg.train_tasks, 1):
            seen.append(task)
            if task_id <= last_done:
                print(f"[Resume] skipping {task} (already done)")
                continue

            self._train_one_task(task, task_id)
            self._eval_tasks(self.cfg.train_tasks, row_key=task, split="test")

            scores = {t: self.score_matrix[task][t] for t in seen}
            aa = average_accuracy(scores)
            bwt = backward_transfer(self.score_matrix, seen)
            fwt = forward_transfer(self.zero_shot_scores, self.score_matrix, seen)

            self.history.append({
                "task_id": task_id,
                "task_name": task,
                "seen_scores": scores,
                "average_accuracy": aa,
                "bwt": bwt,
                "fwt": fwt,
            })

            self._save_history()
```

```
            self._save_score_matrix()

            self.logger.log({
                "event": "task_end",
                "task": task,
                "aa": aa,
                "bwt": bwt,
                "fwt": fwt,
            })
            print(f"[seq_lora {task}] AA={aa:.4f}  BWT={bwt:.4f}  FWT={fwt:.4f}")

            if self.cfg.save_every_task:
                save_checkpoint(
                    self.cfg.output_dir,
                    task_id,
                    self.model,
                    extra={
                        "task": task,
                        "aa": aa,
                        "bwt": bwt,
                        "fwt": fwt,
                    },
                )
                self._mark_done(task_id)

        self._finalise()
        return {"history": self.history, "score_matrix": self.score_matrix}

    def _finalise(self):
        self._save_history()
        self._save_score_matrix()
```

A.13 OrthoHist-LoRA Implementation

Listing A.13: src/ortho_hist_lora.py: OrthoHist-LoRA Implementation

```
import math
from typing import Dict, List, Optional, Tuple
import torch
import torch.nn as nn
import torch.nn.functional as F

class OrthoHistLoRALinear(nn.Module):
    def __init__(self, base_layer: nn.Linear, rank=16, alpha=32, dropout=0.0, mt_rank=8):
        super().__init__()
        self.base_layer = base_layer
        self.rank = rank
        self.alpha = alpha
        self.scaling = alpha / rank
        self.mt_rank = mt_rank
        self.in_features = base_layer.in_features
        self.out_features = base_layer.out_features

        for p in self.base_layer.parameters():
            p.requires_grad_(False)

        self.dropout = nn.Dropout(dropout) if dropout > 0.0 else nn.Identity()

```

```
        self.lora_A = nn.Parameter(torch.empty(rank, self.in_features))
        self.lora_B = nn.Parameter(torch.empty(self.out_features, rank))
        self._init_active_fresh()

        # pool stores alternating (basis [d_out, r], coeff [r, d_in]) as frozen params
        self.basis_pool = nn.ParameterList()
        self._n_pool = 0

        # current task's mixing matrix M_t = U_mt @ V_mt.T
        self.mt_U: Optional[nn.Parameter] = None
        self.mt_V: Optional[nn.Parameter] = None

        # keep archived M_t's for analysis
        # Store the Mt for each task (does not count as a parameter, the true parameters consist solely of the basis pool, the active zone, and the current Mt, not including all Mts)
        self._frozen_mt: List[Tuple[Optional[torch.Tensor], Optional[torch.Tensor]]] = []

        self._hook = self.register_full_backward_hook(self._ortho_grad_hook)

    def _init_active_fresh(self):
        # standard LoRA init for task 1 kaiming: suitable for deep neural networks
        nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
        nn.init.zeros_(self.lora_B)

    def init_active_projected(self):
      if self._n_pool == 0:
          nn.init.kaiming_uniform_(self.lora_A, a=math.sqrt(5))
          nn.init.zeros_(self.lora_B)
          return

      device = self.lora_B.device
      dtype = self.lora_B.dtype
      pool_B = self._cat_pool_basis().float()
      # proj0pool
      #  QR
      B_rand = torch.randn(self.out_features, self.rank, device=device).float()
      # QR
      B_proj = B_rand - pool_B @ (pool_B.T @ B_rand)
      # mode reduced
      q, _ = torch.linalg.qr(B_proj, mode="reduced")

      with torch.no_grad():
          self.lora_B.copy_(q.to(dtype=dtype, device=device))

      nn.init.zeros_(self.lora_A)


    def init_mt(self):
      r_total = self._n_pool * self.rank

      if r_total == 0:
          self.mt_U = None
          self.mt_V = None
          return

      device = self.lora_B.device
      dtype  = self.lora_B.dtype

      U_new = torch.empty(
          r_total,
          self.mt_rank,
```

```
            device=device,
            dtype=dtype,
        )

        V_new = torch.empty(
            r_total,
            self.mt_rank,
            device=device,
            dtype=dtype,
        )
        #  AB
        nn.init.normal_(U_new, mean=0.0, std=0.001)
        nn.init.normal_(V_new, mean=0.0, std=0.001)

        self.mt_U = nn.Parameter(U_new)
        self.mt_V = nn.Parameter(V_new)

    # check
    def forward(self, x):
        base = self.base_layer(x)
        x_d = self.dropout(x)

        active = F.linear(F.linear(x_d, self.lora_A), self.lora_B)

        pool_out = x.new_zeros(active.shape)
        if self._n_pool > 0:
            B_pool = self._cat_pool_basis()   # [d_out, r_total]
            C_pool = self._cat_pool_coeff()   # [r_total, d_in]
            mid = F.linear(x_d, C_pool)       # [..., r_total]

            if self.mt_U is not None:
                # apply (I + U_mt V_mt^T) to mid
                mid = mid + (mid @ self.mt_U) @ self.mt_V.T

            pool_out = F.linear(mid, B_pool)

        return base + self.scaling * (active + pool_out)

    def _cat_pool_basis(self):
        return torch.cat([self.basis_pool[2 * i] for i in range(self._n_pool)], dim=1)

    def _cat_pool_coeff(self):
        return torch.cat([self.basis_pool[2 * i + 1] for i in range(self._n_pool)], dim=0)

    def commit_to_pool(self):
        B_trained = self.lora_B.detach().clone().float()   # [d_out, r]
        A_trained = self.lora_A.detach().clone().float()   # [r, d_in]

        # mode="reduced" Q: [d_out, r], R: [r, r]
        Q, R = torch.linalg.qr(B_trained, mode="reduced")

        # coeff = R @ A, Q @ (R@A) = B @ A
        RA = R @ A_trained  # [r, d_in]

        dtype = self.lora_B.dtype
        device = self.lora_B.device

        # commitfalse!
        self.basis_pool.append(
            nn.Parameter(Q.to(dtype=dtype, device=device), requires_grad=False)
        )
```

```
            self.basis_pool.append(
                nn.Parameter(RA.to(dtype=dtype, device=device), requires_grad=False)
            )
            self._n_pool += 1

        def archive_and_freeze_mt(self):
            U = self.mt_U.detach().clone() if self.mt_U is not None else None
            V = self.mt_V.detach().clone() if self.mt_V is not None else None
            self._frozen_mt.append((U, V))
            # self.mt_U:mt
            if self.mt_U is not None:
                self.mt_U.requires_grad_(False)
                self.mt_V.requires_grad_(False)

        def zero_active_zone(self):
            with torch.no_grad():
                self.lora_A.zero_()
                self.lora_B.zero_()

        @torch.no_grad()
        def _ortho_grad_hook(self, module, grad_input, grad_output):
            # project grad_B away from all pool bases to enforce inter-task orthogonality
            if self.lora_B.grad is None or self._n_pool == 0:
                return
            g = self.lora_B.grad.float()
            for i in range(self._n_pool):
                b = self.basis_pool[2 * i].detach().float()
                g = g - b @ (b.T @ g)
            self.lora_B.grad.copy_(g.to(dtype=self.lora_B.grad.dtype))

        #
        def mt_block_norms(self):
            # Frobenius norm of each row-block of M_t; block i corresponds to historical task i+1
            if self.mt_U is None or self._n_pool == 0:
                return None
            M = (self.mt_U @ self.mt_V.T).detach().float()
            # row represents reuse
            return [
                M[i * self.rank:(i + 1) * self.rank, :].norm().item()
                for i in range(self._n_pool)
            ]


    class OrthoHistManager:
        def __init__(self, model, target_modules, rank=16, alpha=32, dropout=0.0, mt_rank=8, device=None):
            self.model = model
            self.target_modules = tuple(target_modules)
            self.rank = rank
            self.alpha = alpha
            self.mt_rank = mt_rank
            self.device = device or next(model.parameters()).device
            self.current_task_id = 0
            self.layers: Dict[str, OrthoHistLoRALinear] = {}
        # 1.
        def inject(self):
            dtype = next(self.model.parameters()).dtype
            replaced = {}
            for name, module in list(self.model.named_modules()):
                if not isinstance(module, nn.Linear):
                    continue
                if not any(name.endswith(t) for t in self.target_modules):
```

```
                continue
            parent_name, child_name = name.rsplit(".", 1)
            parent = self.model.get_submodule(parent_name)
            wrapper = OrthoHistLoRALinear(
                module, rank=self.rank, alpha=self.alpha, mt_rank=self.mt_rank
            ).to(device=self.device, dtype=dtype)
            setattr(parent, child_name, wrapper)
            replaced[name] = wrapper

        if not replaced:
            raise RuntimeError(f"No target modules found. target_modules={self.target_modules}")
        self.layers = replaced

    # 2. initialization
    def prepare_for_task(self, task_id):
        self.current_task_id = task_id
        for name, layer in self.layers.items():
            if task_id == 1:
                layer._init_active_fresh()
            else:
                layer.init_active_projected()
            layer.init_mt()
        self._set_trainable()

    # 3. set requires grad
    def _set_trainable(self):
        for p in self.model.parameters():
            p.requires_grad_(False)
        for layer in self.layers.values():
            layer.lora_A.requires_grad_(True)
            layer.lora_B.requires_grad_(True)
            if layer.mt_U is not None:
                layer.mt_U.requires_grad_(True)
                layer.mt_V.requires_grad_(True)
    # 4.
    def trainable_parameters(self):
        params, seen = [], set()
        for layer in self.layers.values():
            for p in [layer.lora_A, layer.lora_B, layer.mt_U, layer.mt_V]:
              # id(p): parameter's address
                if p is not None and p.requires_grad and id(p) not in seen:
                    params.append(p)
                    seen.add(id(p))
        return params

    # 5
    def commit_task(self):
        for name, layer in self.layers.items():
            # commit B and A
            layer.commit_to_pool()
            # commit M_t
            layer.archive_and_freeze_mt()
            layer.zero_active_zone()

    def n_trainable_params(self):
        return sum(p.numel() for p in self.trainable_parameters())

    def summary(self):
        n_pool = next(iter(self.layers.values()))._n_pool
        return {
            "method": "ortho_hist",
```

```
            "task_id": self.current_task_id,
            "n_pool": n_pool,
            "r_total": n_pool * self.rank,
            "mt_rank": self.mt_rank,
            "trainable_params": self.n_trainable_params(),
        }

    # 3 Mt  commit  collect
    def collect_reuse_analysis(self, task_name):
        result = {"task_name": task_name, "layers": {}}
        for name, layer in self.layers.items():
            norms = layer.mt_block_norms()
            result["layers"][name] = norms if norms is not None else []
        return result
```

A.14 Checkpointing Utilities

Listing A.14: src/checkpointing.py: Checkpointing Utilities

```
from __future__ import annotations

import json
import os
from pathlib import Path
from typing import Any, Dict, Optional

import torch

def _json_safe(value):
    if isinstance(value, (str, int, float, bool)) or value is None:
        return value
    if isinstance(value, list):
        return [_json_safe(v) for v in value]
    if isinstance(value, dict):
        return {str(k): _json_safe(v) for k, v in value.items()}
    return str(value)


def _save_json_atomic(payload: Dict[str, Any], path: Path) -> None:
    tmp = path.with_suffix(path.suffix + ".tmp")
    with open(tmp, "w", encoding="utf-8") as f:
        json.dump(_json_safe(payload), f, indent=2, ensure_ascii=False)
    os.replace(tmp, path)


def save_checkpoint(
    output_dir,
    task_id,
    model,
    extra: Optional[Dict[str, Any]] = None,
):
    ckpt_dir = Path(output_dir) / "checkpoints" / f"task_{task_id:02d}"
    ckpt_dir.mkdir(parents=True, exist_ok=True)

    # Save the full model state. For OrthoHist-LoRA this includes the frozen
    # historical basis_pool; for Sequential LoRA it includes the PEFT LoRA state.
    state = model.state_dict()
    tmp_path = ckpt_dir / "model_state.pt.tmp"
```

```
    final_path = ckpt_dir / "model_state.pt"
    torch.save(state, tmp_path)
    os.replace(tmp_path, final_path)

    if extra:
        _save_json_atomic(extra, ckpt_dir / "meta.json")


def load_checkpoint(output_dir, task_id, model):
    ckpt_dir = Path(output_dir) / "checkpoints" / f"task_{task_id:02d}"
    ckpt_path = ckpt_dir / "model_state.pt"

    if not ckpt_path.exists():
        raise FileNotFoundError(f"Checkpoint not found: {ckpt_path}")

    state = torch.load(ckpt_path, map_location="cpu")
    incompatible = model.load_state_dict(state, strict=False)

    missing = list(getattr(incompatible, "missing_keys", []))
    unexpected = list(getattr(incompatible, "unexpected_keys", []))

    if unexpected:
        print(f"[Checkpoint WARNING] unexpected keys while loading {ckpt_path}:")
        for key in unexpected[:20]:
            print(f"  - {key}")
        if len(unexpected) > 20:
            print(f"  ... and {len(unexpected) - 20} more")

    if missing:
        print(f"[Checkpoint WARNING] missing keys while loading {ckpt_path}:")
        for key in missing[:20]:
            print(f"  - {key}")
        if len(missing) > 20:
            print(f"  ... and {len(missing) - 20} more")

    return incompatible
```

A.15 Analysis and Visualization Utilities

Listing A.15: src/analysis_utils.py: Analysis and Visualization Utilities

```
from __future__ import annotations

import json
from pathlib import Path

import numpy as np


def collect_reuse_matrix(reuse_snapshots, task_names):
    T = len(task_names)
    matrix = np.full((T, T), np.nan)
    task_to_idx = {name: idx for idx, name in enumerate(task_names)}

    for snap in reuse_snapshots:
        task_name = snap.get("task_name")
        if task_name not in task_to_idx:
            continue
```

```
        t_idx = task_to_idx[task_name]
        if t_idx == 0:
            continue

        layer_data = snap.get("layers", {})
        layer_means = []
        for norms in layer_data.values():
            if norms and len(norms) == t_idx:
                layer_means.append(norms)

        if not layer_means:
            continue

        arr = np.array(layer_means, dtype=float)       # [n_layers, t_idx]
        mean_per_hist = arr.mean(axis=0)               # [t_idx]
        for i, val in enumerate(mean_per_hist):
            matrix[t_idx, i] = val

    return matrix


def save_reuse_heatmap(matrix, task_names, output_dir):
    out = Path(output_dir) / "analysis"
    out.mkdir(parents=True, exist_ok=True)

    serialisable = {
        task_names[t]: {
            task_names[i]: float(matrix[t, i])
            for i in range(t)
            if not np.isnan(matrix[t, i])
        }
        for t in range(len(task_names))
    }

    with open(out / "reuse_matrix.json", "w", encoding="utf-8") as f:
        json.dump(serialisable, f, indent=2, ensure_ascii=False)

    try:
        import matplotlib.pyplot as plt

        fig, ax = plt.subplots(figsize=(9, 7))
        masked = np.ma.masked_invalid(matrix)
        im = ax.imshow(masked, aspect="auto", cmap="YlOrRd")
        plt.colorbar(im, ax=ax, label="Mean M_t block norm (Frobenius)")
        ax.set_xticks(range(len(task_names)))
        ax.set_yticks(range(len(task_names)))
        ax.set_xticklabels(task_names, rotation=45, ha="right", fontsize=8)
        ax.set_yticklabels(task_names, fontsize=8)
        ax.set_xlabel("Historical task")
        ax.set_ylabel("Current task")
        ax.set_title("M_t reuse strength")

        for t in range(len(task_names)):
            for i in range(len(task_names)):
                v = matrix[t, i]
                if not np.isnan(v):
                    ax.text(i, t, f"{v:.2f}", ha="center", va="center", fontsize=7)

        plt.tight_layout()
        plt.savefig(out / "reuse_heatmap.png", dpi=150)
```

```
        plt.close()
    except Exception as e:
        print(f"[analysis] could not save reuse heatmap: {e}")


def save_score_matrix(score_matrix, task_names, output_dir):
    out = Path(output_dir) / "analysis"
    out.mkdir(parents=True, exist_ok=True)

    with open(out / "score_matrix.json", "w", encoding="utf-8") as f:
        json.dump(score_matrix, f, indent=2, ensure_ascii=False)

    try:
        import matplotlib.pyplot as plt

        T = len(task_names)
        mat = np.full((T, T), np.nan)

        for r_idx, r_name in enumerate(task_names):
            row = score_matrix.get(r_name, {})
            for c_idx, c_name in enumerate(task_names):
                if c_name in row:
                    mat[r_idx, c_idx] = row[c_name]

        fig, ax = plt.subplots(figsize=(9, 7))
        im = ax.imshow(np.ma.masked_invalid(mat), aspect="auto", cmap="Blues", vmin=0, vmax=1)
        plt.colorbar(im, ax=ax, label="Score")
        ax.set_xticks(range(T))
        ax.set_yticks(range(T))
        ax.set_xticklabels(task_names, rotation=45, ha="right", fontsize=8)
        ax.set_yticklabels(task_names, fontsize=8)
        ax.set_xlabel("Evaluated task")
        ax.set_ylabel("Trained up to task")
        ax.set_title("Continual learning score matrix")

        for r in range(T):
            for c in range(T):
                v = mat[r, c]
                if not np.isnan(v):
                    ax.text(c, r, f"{v:.2f}", ha="center", va="center", fontsize=7)

        plt.tight_layout()
        plt.savefig(out / "score_matrix.png", dpi=150)
        plt.close()
    except Exception as e:
        print(f"[analysis] could not save score matrix: {e}")


def count_trainable_params(model):
    trainable = sum(p.numel() for p in model.parameters() if p.requires_grad)
    total = sum(p.numel() for p in model.parameters())
    return {"trainable": trainable, "total": total}
```

Appendix B

Appendix B: User Manual

B.1 Quick Start

Listing B.1: Quick start.

```
git clone PUT-GITHUB-REPOSITORY-URL-HERE
cd ECE-5996-design-proj-main
pip install -r requirements.txt
python main.py --config configs/sequential_lora.yaml
python main.py --config configs/ortho_hist_k4.yaml
```

B.2 Reading Final Metrics

Listing B.2: Read final metrics from a run.

```
import json
from pathlib import Path

run = Path("outputs/ortho_hist_k4")
hist = json.load(open(run / "history.json", "r", encoding="utf-8"))
last = hist[-1]
print(last["average_accuracy"])
print(last["bwt"])
print(last["seen_scores"])
```